

UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO – BCC

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

JOINVILLE

2025

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Adriano Fiorese

JOINVILLE

2025

PEDRO FARIA FERNANDES

**OTIMIZAÇÃO DE BALANCEADORES DE CARGA DE SISTEMAS DE
DISTRIBUIÇÃO DE CONTEÚDO POR MEIO DE MONITORAMENTO
SISTEMÁTICO DE MEMÓRIA**

Trabalho de conclusão de curso submetido à Universidade do Estado de Santa Catarina como parte dos requisitos para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Adriano Fiorese

Orientador:

BANCA EXAMINADORA:

Dr. Adriano Fiorese
UDESC

Membros:

Dra. Débora Cabral Nazário
UDESC

M.e. Paulo Roberto Vieira Jr.
IFPR

Joinville, 01 de maio de 2025

RESUMO

No século XXI, sistemas digitais na internet se consolidaram como principal meio de consumo de conteúdo em áudio e vídeo. Em decorrência disso, projetistas têm tido como desafio a realização da otimização contínua desses sistemas de modo a garantir que eles ofereçam a melhor QoE (Quality of Experience) ao usuário final, sobretudo em meio à restrição econômica de compatibilidade com a tecnologia já implementada. Devido ao grande volume de dados sendo trafegados, pode-se inferir que há padrões de consumo de conteúdo específicos e heterogêneos, variando de acordo com as especificações e contextos de cada sistema. Dessa forma, sabe-se que os balanceadores de carga são elementos-chave no processo de roteamento de requisições e, portanto, sua otimização é de grande valia para o processo de otimização de conteúdo. Portanto, o presente trabalho propõe a elaboração, implementação e posterior análise de um algoritmo balanceador de carga especificamente para servidores de distribuição de conteúdo com base em monitoramento de memória principal e secundária.

Palavras-chave: Balanceamento de Carga. Sistemas Distribuídos. Distribuição de Conteúdo. Monitoramento de Memória. Streaming de Vídeo. CDN. Cache de Servidores. Arquitetura de Sistemas. QoE. Redes de Computadores. Heurísticas de Balanceamento.

ABSTRACT

In the 21st century, digital systems on the internet have established themselves as the primary means for consuming audio and video content. Consequently, designers have faced the challenge of continuously optimizing these systems to ensure they offer the best Quality of Experience (QoE) to the end-user, especially amidst the economic constraint of maintaining compatibility with already implemented technology. Due to the large volume of data being transferred, it can be inferred that there are specific and heterogeneous content consumption patterns, varying according to the specifications and contexts of each system. Thus, it is known that load balancers are key elements in the request routing process, and therefore, their optimization is of great value to the content optimization process. Therefore, this work proposes the design, implementation, and subsequent analysis of a load balancing algorithm specifically for content delivery servers based on primary and secondary memory monitoring.

Keywords: Load Balancing. Distributed Systems. Content Delivery. Memory Monitoring. Video Streaming. CDN. Server Caching. Systems Architecture. QoE. Computer Networks. Balancing Heuristics.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo detalhado das múltiplas camadas de <i>caching</i> em CDNs.	24
Figura 2 – Arquitetura geral do sistema.	37
Figura 3 – Aplicação cliente MPEG-DASH com visualização em tempo real de métricas de QoE: gráfico de qualidade, gráfico de <i>bitrate</i> e gráfico de travamentos. . .	47
Figura 4 – Painel de monitoramento em tempo real mostrando métricas de consumo de memória, taxa de leitura de disco e requisições ativas dos servidores MPEG-DASH.	49

LISTA DE ABREVIATURAS E SIGLAS

ALB	Application Load Balancer
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AWS	Amazon Web Services
BGP	Border Gateway Protocol
CDN	Content Delivery Network (Rede de Entrega de Conteúdo)
CPU	Central Processing Unit (Unidade Central de Processamento)
DASH	Dynamic Adaptive Streaming over HTTP
DLQ	Dead Letter Queue
DNS	Domain Name System (Sistema de Nomes de Domínio)
gRPC	Google Remote Procedure Call
InP	Provedor de Infraestrutura
IoT	Internet of Things (Internet das Coisas)
ISP	Internet Service Provider (Provedor de Serviço de Internet)
JSON	JavaScript Object Notation
LXC	Linux Containers
MPD	Media Presentation Description
MPEG	Moving Picture Experts Group
MQTT	Message Queuing Telemetry Transport
NGINX	Software utilizado como servidor web, proxy reverso e balanceador de carga
NIC	Network Interface Card (Placa de Rede)
NS	Network Simulator (utilizado no contexto do NS-3)
OSI	Open Systems Interconnection
PoP	Point of Presence (Ponto de Presença)
QoE	Quality of Experience (Qualidade de Experiência)
REST	Representational State Transfer
SDN	Software-Defined Network (Rede Definida por Software)
SFTP	Secure File Transfer Protocol
SOAP	Simple Object Access Protocol
SP	Provedor de Serviço

SSL	Secure Sockets Layer
TCP	Transmission Control Protocol (Protocolo de Controle de Transmissão)
URL	Uniform Resource Locator
VETH	Virtual Ethernet
VMM	Virtual Machine Monitor (Monitor de Máquina Virtual)
VMs	Virtual Machines (Máquinas Virtuais)
WAF	Web Application Firewall
WSDL	Web Services Description Language
XML	Extensible Markup Language

SUMÁRIO

1	INTRODUÇÃO	10
1.1	O PROBLEMA	10
1.2	TRABALHOS RELACIONADOS	12
1.3	JUSTIFICATIVA	14
1.4	OBJETIVOS	15
1.5	METODOLOGIA E ESTRUTURA	15
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	BALANCEADORES DE CARGA	16
2.1.1	Balanceadores de carga em nível de aplicação	17
2.1.2	Balanceadores de carga em nível de rede	18
2.2	MPEG-DASH	19
2.2.1	<i>Content Steering</i>	20
2.3	SERVIDORES DE DISTRIBUIÇÃO DE CONTEÚDO E CDNS	21
2.3.1	Funcionamento de cache em redes de entrega de conteúdo	22
2.4	MONITORAMENTO DE MEMÓRIA	25
2.4.1	APIs fornecidas pelos principais sistemas operacionais	25
2.4.2	Aplicações	25
2.5	TRANSMISSÃO CONTÍNUA DE DADOS	26
2.6	INTERPRETAÇÃO DE DADOS OBSOLETOS	27
2.6.1	Algoritmos de Interpretação de Carga (Load Interpretation - LI)	28
2.6.2	Desempenho e Conclusões	28
2.7	FILAS DE MENSAGENS PARA COMUNICAÇÃO EM SISTEMAS DISTRIBUÍDOS	29
2.8	VIRTUALIZAÇÃO DE REDES PARA SIMULAÇÃO DE CENÁRIOS	30
2.8.1	Virtualização baseada em Containers Linux	30
2.8.2	Simulação de Redes Definidas por Software (SDN)	31
3	PROPOSTA	32
3.1	O ALGORITMO	32
3.2	OS REQUISITOS TECNOLÓGICOS	35
3.3	ANÁLISE COMPARATIVA	37
4	DESCRIÇÃO DO EXPERIMENTO	39
4.1	AMBIENTE DE EXPERIMENTAÇÃO	39
4.1.1	Particularidades do Docker e cgroups	39
4.1.2	Organização do conteúdo e bind mounts	40
4.2	DEFINIÇÃO DOS MÓDULOS	40

4.2.1	Módulos Principais	40
4.2.2	Módulos Auxiliares	41
4.3	DETALHES DE IMPLEMENTAÇÃO DOS MÓDULOS	41
4.3.1	Servidores MPEG-DASH	42
<i>4.3.1.1</i>	<i>Monitoramento e Publicação de Métricas</i>	<i>42</i>
4.3.2	Balanceador de Carga	43
<i>4.3.2.1</i>	<i>Arquitetura e Funcionamento</i>	<i>43</i>
<i>4.3.2.2</i>	<i>Estratégias de Balanceamento Implementadas</i>	<i>43</i>
4.3.3	Gerador de Carga	44
<i>4.3.3.1</i>	<i>Simulação de Clientes MPEG-DASH</i>	<i>44</i>
<i>4.3.3.2</i>	<i>Aplicação Cliente para Validação e Visualização</i>	<i>45</i>
4.3.4	Broker MQTT (NanoMQ)	46
4.3.5	Orquestração com Docker Compose	48
4.3.6	Painel de Monitoramento	48
4.4	GERAÇÃO DE CONTEÚDO	49
4.4.1	Codificação e Segmentação MPEG-DASH	49
4.4.2	Replicação e Distribuição do Conteúdo	51
4.5	CENÁRIOS DE TESTE	51
4.5.1	Parâmetros Globais	52
4.5.2	Definição dos Cenários	52
4.5.3	Descrição dos Cenários	52
<i>4.5.3.1</i>	<i>Cenário 1: Baixa Concorrência com Servidores Heterogêneos</i>	<i>52</i>
<i>4.5.3.2</i>	<i>Cenário 2: Baixa Concorrência com Servidores Homogêneos</i>	<i>53</i>
<i>4.5.3.3</i>	<i>Cenário 3: Alta Concorrência com Servidores Heterogêneos</i>	<i>53</i>
<i>4.5.3.4</i>	<i>Cenário 4: Alta Concorrência com Servidores Homogêneos</i>	<i>53</i>
4.5.4	Estratégias de Balanceamento Avaliadas	53
4.5.5	Métricas de Avaliação	54
5	CONSIDERAÇÕES PARCIAIS	55
5.1	PRÓXIMOS PASSOS	55
5.2	CRONOGRAMA	56
	REFERÊNCIAS	57
	GLOSSÁRIO	61

1 INTRODUÇÃO

Consolidada como um produto extremamente popular, a televisão é o dispositivo central na maioria das salas de estar da população mundial. Em decorrência disso, a população se adaptou à prática de consumo de conteúdo em vídeo, criando uma base de consumidores preparada para a revolução do entretenimento digital. Com a popularização da Internet e o surgimento das plataformas de *streaming*, esse hábito de consumo foi potencializado, migrando para um modelo sob demanda. Essa mudança gerou um crescimento exponencial no volume de dados de vídeo trafegando pela rede, impondo uma pressão sem precedentes sobre a infraestrutura global e tornando o balanceamento de tráfego uma necessidade crítica para garantir a qualidade do serviço aos usuários finais.

Desse modo, pode-se constatar que há desafios no contexto da otimização contínua dos processos de distribuição de conteúdo sob demanda, haja vista que é necessário maximizar a Qualidade de Experiência (QoE) oferecida pelos serviços para cada vez mais usuários. Contudo, é crucial reconhecer que a percepção de qualidade de experiência pelo usuário final (QoE) não depende unicamente da infraestrutura e capacidade dos provedores de serviço. Ela é, na verdade, uma responsabilidade compartilhada, na qual os provedores de acesso à Internet (ISPs) desempenham um papel igualmente fundamental. Frequentemente, a infraestrutura de rede dos ISPs não acompanha a evolução dos provedores de serviço e, por isso, a qualidade percebida pelo usuário pode não ser ótima. O objetivo principal deste trabalho não será a otimização absoluta da QoE, mas sim a otimização da qualidade no contexto do provedor de serviço.

A distribuição de conteúdo de vídeo em larga escala geralmente ocorre por meio de uma arquitetura complexa e geograficamente dispersa, conhecida como Rede de Distribuição de Conteúdo (CDN - *Content Delivery Network*). Nessa estrutura, o conteúdo original é replicado do servidor de origem para múltiplos servidores de borda (*edge servers*) localizados em pontos estratégicos, mais próximos dos usuários finais. Quando um espectador solicita um vídeo, a requisição é direcionada ao servidor de borda geograficamente mais próximo, diminuindo a latência e o tempo de carregamento. Para otimizar esse processo e lidar com picos de audiência e a vasta quantidade de solicitações simultâneas, o balanceamento de carga torna-se indispensável. Ele distribui o tráfego de forma inteligente entre os diversos servidores de conteúdo de vídeo disponíveis, evitando que um único ponto da rede fique sobrecarregado. Essa distribuição equitativa é fundamental para prevenir gargalos, garantir a estabilidade do serviço e, consequentemente, otimizar a experiência de visualização para os consumidores.

1.1 O PROBLEMA

Em sistemas distribuídos modernos caracterizados pela replicação de serviços, o balanceador de carga é peça fundamental no processo de maximização do aproveitamento de recursos computacionais e entende-se que sua efetividade está na capacidade de não sobrecarregar nenhum serviço mesmo em cenários de alta demanda (Passos; Fiorese, 2019). De modo

simplificado, seu funcionamento consiste em, além de centralizar o roteamento de requisições, oferecer uma abstração para os consumidores, distribuir a carga entre vários servidores de modo a evitar sobrecarga em um ou mais deles. Nesse aspecto, podem ser empregadas várias estratégias para realizar esse roteamento: Seleção aleatória, *Round-Robin*, eleição de sistemas distribuídos, dentre outros. Enquanto são as mais utilizadas, a seleção aleatória e *Round-Robin* são heurísticas simples de serem implementadas, mas que podem eventualmente causar sobrecarga em um ou mais servidores. A abordagem que contorna esse problema é justamente a eleição do servidor mais apropriado para lidar com a requisição de acordo com parâmetros personalizados que evidenciam, usando alguma estratégia, qual é o servidor a ser escolhido. A escolha dessa estratégia é a área-chave da atuação da proposta deste trabalho.

Além disso, dispositivos de armazenamento persistente (também conhecidos como dispositivos de memória secundária) são muitas vezes pontos de gargalo em sistemas, sobretudo sistemas de distribuição de conteúdo em *streaming*, que exigem acesso aleatório à memória (Mei et al., 2013). Esses dispositivos de armazenamento persistente exibem custos arquiteturais altos (Arpaci-Dusseau et al., 1998). Nesse sentido, em Data Centers de larga escala, falhas de dispositivos de memória secundária são a regra e não a exceção (Wu et al., 2018), o que faz com que esses sistemas de alta escalabilidade tenham que ser tolerantes a falhas dos dispositivos de memória secundária. Dessa forma, entende-se que a métrica de taxa de uso de memória secundária é de grande valia no contexto da tomada de decisão de balanceadores de carga de requisições de distribuição de conteúdo. Em decorrência disso, para escalar sistemas na Web de forma sólida, uma técnica muito difundida por projetistas de sistemas é o armazenamento de dados na memória principal. A redundância passou a ser característica de sistemas escaláveis na Web. Isso se deve à evolução do número de usuários finais, ao consequente surgimento do Big Data gerando paradigmas revolucionadores no contexto de Bancos de Dados como os bancos não-relacionais (Tailor Sushant Choudhary, 2014), ao uso em massa das CDNs, à popularização da técnica de virtualização como uma forma de obter confiabilidade nos sistemas, dentre outros. Todos esses fatores corroboraram para que a memória principal, sob o seu principal benefício (a velocidade de acesso), fosse utilizada como método primário de armazenamento de dados persistentes, mesmo com a sua volatilidade intrínseca.

Dessa forma, considerando que o papel dos servidores de distribuição de conteúdo é recuperar e entregar dados (relativamente não envolvendo tanto processamento quanto outros tipos de servidores), é possível conceber que a métrica do uso de memória (principal e secundária) é extremamente correlacionada com o desempenho desses servidores - considerando o contexto de múltiplas instâncias de servidores que mantêm seus próprios meios de armazenamento interno - cenário muito presente em CDNs com os servidores de borda (*Edge Servers* ou *Cache Servers*) (Ali; Fang; Khan, 2025).

Diante do exposto, evidencia-se uma lacuna crítica nas estratégias de balanceamento de carga convencionais. Abordagens como *Round-Robin* ou seleção aleatória, ao ignorarem o estado atual dos recursos de memória, podem inadvertidamente direcionar requisições para

nós computacionais já sobrecarregados, que estejam sofrendo com alta latência de I/O em disco ou próximos da exaustão de memória principal. Essa falha de percepção não só degrada o desempenho, como também aumenta o risco de falhas em cascata. Portanto, o problema central a ser atacado neste trabalho é a otimização da escolha do servidor em balanceadores de carga para sistemas de distribuição de conteúdo, desenvolvendo uma estratégia que incorpore o monitoramento de métricas de memória (primária e secundária) como fator decisório principal. O objetivo é criar um mecanismo de balanceamento mais inteligente e preditivo, capaz de evitar gargalos de desempenho antes que eles impactem o usuário final, aumentando assim a eficiência e a resiliência do sistema.

1.2 TRABALHOS RELACIONADOS

O sistema NGINX é amplamente utilizado como servidor web, proxy reverso e balanceador de carga para lidar com aplicações de alta concorrência. Seus algoritmos nativos de balanceamento, como *Weighted Round-Robin*, *Least Connections* e *IP hash*, são as estratégias mais comuns. A abordagem *IP hash*, por exemplo, foi projetada para direcionar requisições de um mesmo cliente para o mesmo servidor, mantendo a continuidade da sessão. Contudo, a função de *hash* nativa é considerada muito simples, o que leva a uma alta probabilidade de "colisões de hash". Essas colisões resultam em uma distribuição de carga não uniforme, fazendo com que alguns servidores fiquem sobrecarregados enquanto outros permanecem subutilizados (Ma; Chi, 2022). Essa limitação evidencia que, embora os balanceadores de carga baseados no NGINX sejam eficientes em distribuir o tráfego de modo geral, não há uma estratégia que monitore o uso de memória em tempo real para ajudar na tomada de decisões dinâmicas, sobretudo no contexto de distribuição de conteúdo, cenário no qual a recuperação de arquivos em memória é predominante. Isso pode resultar em sobrecarga nos servidores de cache, especialmente em cenários de alto tráfego, onde a memória é um recurso crítico. Portanto, a adição de uma camada de monitoramento de memória pode aprimorar a inteligência do balanceador de carga, permitindo uma alocação de tráfego que considere o estado real dos recursos do servidor, em vez de depender apenas de heurísticas de distribuição.

O *HAProxy* (HAProxy, 2025) é um software de código aberto que fornece balanceamento de carga e proxy reverso para aplicações baseadas em TCP/IP e HTTP. Embora seja reconhecido por sua eficiência e baixo uso de memória, não há indicações de que utilize monitoramento de memória em tempo real para otimizar a distribuição de tráfego. Integrar tal funcionalidade poderia aprimorar a alocação de recursos e prevenir sobrecargas nos servidores de cache.

No contexto da poderosa e moderna prática de *containerização* de servidores, o *Docker* é uma plataforma de código aberto disponibilizada especificamente para esse propósito. O *Docker Swarm*, mecanismo interno da plataforma, realiza a orquestração sistemática de contêineres em sistemas complexos e computacionalmente descentralizados. Embora a ferramenta possua um mecanismo interno de balanceamento de carga, o roteamento é feito utilizando técnicas

simples como *Round-Robin* e distribuição aleatória para realizar o roteamento distribuído de requisições para os servidores em contêineres. Em 2018, foi realizado um experimento utilizando essas ferramentas de modo a introduzir a influência do consumo de memória nos servidores e, consequentemente, aumentar a eficiência do balanceador de carga (Bella; Data; Yahya, 2018). No entanto, a pesquisa não só teve foco exclusivamente voltado às especificidades da ferramenta *Docker*, mas também não envolveu nenhuma análise no contexto de servidores de distribuição de conteúdo.

Também foram elaborados vários algoritmos que tornam o balanceamento de carga dinâmico (*Dynamic Load Balancing*). Um deles (Adewojo; Bass, 2023) teve foco na determinação e uso de métricas cuidadosamente selecionadas para recursos computacionais dos servidores de destino: Número de *threads*, tamanho do *buffer* de rede, consumo de CPU, consumo de memória e largura de banda. O objetivo do trabalho é propor uma nova técnica para a resolução do problema causado pelo fenômeno recorrente na *web* chamado de *Flash Crowds*, caracterizado pelo aumento abrupto de requisições de clientes em serviços *web*. O trabalho trata de aplicações *cloud* no geral, não tendo especificidade voltada a servidores de distribuição de conteúdo.

Uma revisão (Mahmood et al., 2021) foi realizada de modo a obter um retrato do panorama geral das técnicas mais utilizadas para o balanceamento de carga. Após a apresentação dos algoritmos, uma comparação sistemática foi realizada entre eles e um dos critérios de comparação foi a parametrização utilizada por cada algoritmo. Quase todos os algoritmos apresentados pela revisão são dinâmicos e utilizam mais de um parâmetro de configuração. Do total de 15 algoritmos apresentados, 11 usam o tempo de resposta da requisição como parâmetro, 9 utilizam a vazão de dados, 6 utilizam consumo de memória nos servidores, 6 utilizam o consumo de disco, 5 usam o consumo de recursos computacionais em geral na unidade do servidor, 4 usam especificamente o consumo de CPU, 4 usam como parâmetro a duração média das requisições (que se diferencia do tempo de resposta, visto que este é monitorado individualmente por requisições de teste que não refletem a carga verdadeira), 2 usam técnicas simples como *Round-Robin* e distribuição aleatória, 1 usa a taxa de falha de requisições e 1 usa o número de conexões simultaneamente ativas. Dessa forma, percebe-se que, na revisão, não há nenhum algoritmo que possivelmente fundamentaria a criação de uma arquitetura de balanceamento de carga especificamente voltada para servidores de distribuição de conteúdo, tendo como base exclusivamente o monitoramento de memória em servidores.

No trabalho "*Dynamic Load Balancing for Efficient Video Streaming Service*" (Kim; Won, 2015), é abordado o desequilíbrio de carga em servidores de *streaming* causado pela concentração de requisições em conteúdos populares. A estratégia proposta é um sistema de gerenciamento de conteúdo dinâmico, denominado iSCON, que monitora os servidores em tempo real e redistribui a carga por meio da replicação de conteúdos populares em servidores com menor tráfego. A abordagem dos autores se concentra exclusivamente na largura de banda de transmissão como o principal indicador de carga; o algoritmo de balanceamento é acionado quando a largura de banda de um servidor excede um limiar predefinido. Embora as especificações dos servidores, incluindo

a memória RAM, sejam detalhadas, o monitoramento do uso de memória não é considerado um parâmetro para as decisões de balanceamento de carga. O foco permanece na equalização do tráfego de rede, deixando uma lacuna na otimização de cenários onde a pressão sobre a memória, e não apenas a saturação da largura de banda, pode se tornar o gargalo para a performance do sistema de distribuição de conteúdo.

No trabalho "*Data Allocation and Dynamic Load Balancing for Distributed Video Storage Server*", (Tsao et al., 1999) propõem uma solução de duas fases para aumentar a disponibilidade em um *cluster* de servidores de vídeo distribuídos. A primeira fase consiste em um esquema de alocação inicial de dados que distribui réplicas de vídeo pelos servidores com base em probabilidades de acesso esperadas, a fim de alcançar um balanceamento de carga estático. A segunda fase é uma estratégia de balanceamento de carga dinâmica chamada "*load shifting*" (deslocamento de carga), na qual uma requisição bloqueada em um servidor sobrecarregado pode ser admitida ao se migrar uma ou mais requisições já em andamento daquele servidor para outros nós no *cluster*, potencialmente em uma reação em cadeia. O acionamento deste mecanismo dinâmico é baseado em limiares predefinidos que comparam a carga de um servidor, medida pelo número de *streams* ativas ($P_{i,j}$), com a carga de seus vizinhos ou com sua capacidade máxima de serviço (A_{max}). O conceito de "carga" é, portanto, uma função do número de *streams* ativas em relação à capacidade máxima do servidor, sem considerar o estado de recursos de mais baixo nível, como o uso de memória ou CPU. Esta abstração é eficaz para balancear o número de requisições, mas não endereça potenciais gargalos de desempenho que podem surgir do esgotamento da memória do sistema, especialmente em servidores que realizam cache intensivo de dados de vídeo, um problema que o presente estudo visa investigar.

1.3 JUSTIFICATIVA

A eficiência dos Sistemas de Distribuição de Conteúdo (CDNs) é intrinsecamente relacionada à capacidade de seus servidores em entregar dados armazenados estaticamente de forma rápida aos usuários finais. Nesse contexto, a memória - tanto a principal utilizada em cache quanto em disco - emerge como um recurso crítico, cujo estado reflete diretamente a aptidão de um servidor para responder a novas requisições de conteúdo.

Portanto, a proposta da elaboração e implementação de um algoritmo de balanceamento de carga justifica-se pela necessidade premente de explorar uma estratégia mais especializada e diretamente alinhada com a natureza operacional dos servidores de distribuição de conteúdo. Uma abordagem que utilize o monitoramento sistemático e exclusivo de memória dos servidores como critério para distribuição de carga visa preencher uma lacuna significativa e acredita-se que tal abordagem pode resultar em um sistema mais eficiente, capaz de otimizar a utilização de recursos de cache, prevenir a sobrecarga dos servidores de forma mais eficaz e, conseqüentemente, aprimorar a latência e a experiência do usuário final na entrega do conteúdo.

1.4 OBJETIVOS

Objetivo geral: Elaborar, implementar e analisar um algoritmo de balanceamento de carga para servidores de distribuição de conteúdo com base em monitoramento de memória de servidor de distribuição de conteúdo, comparando-o com outros algoritmos da literatura;

Objetivos Específicos:

- Estudar as principais arquiteturas usadas em servidores de cache de CDNs;
- Estudar os principais modelos de persistência de dados em servidores de distribuição de conteúdo;
- Revisar métodos de monitoramento de memória secundária;
- Revisar métodos de monitoramento de memória principal;
- Listar requisitos topológicos da rede e da estrutura dos servidores que definirão o contexto da aplicação da metodologia;
- Levantar estratégias de transmissão contínua de dados de monitoramento de servidores de distribuição de conteúdo para o serviço de balanceador de carga;
- Estabelecer, conceitualmente, algoritmo de escolha de servidores (por parte do balanceador de carga) com base nos dados históricos fornecidos;
- Implementar o modelo criado com um balanceador de carga simples;
- Avaliar os resultados obtidos, tendo como principal métrica a eficiência de escolha do balanceador de carga, por meio de testes de carga;

1.5 METODOLOGIA E ESTRUTURA

Este trabalho consiste na realização das três principais tarefas a seguir: Apresentação e contextualização do problema; Realização de uma revisão de literatura com os objetivos de fundamentar teoricamente o tema e fornecer uma compreensão sólida; Proposição de um modelo de implementação com todos os detalhes técnicos e conceituais necessários. A organização das atividades se dá pelas seções de Introdução, Fundamentação Teórica e Proposta, respectivamente, conforme a ordem das atividades citadas.

2 FUNDAMENTAÇÃO TEÓRICA

Para embasar a proposta central deste trabalho, este capítulo se dedica a revisar os conceitos fundamentais. A exploração detalhada destes pilares teóricos é essencial para contextualizar as decisões metodológicas, o planejamento e a implementação subsequentes e a análise dos resultados, conferindo uma base sólida e coesa ao desenvolvimento da pesquisa.

Serão abordados, inicialmente, os balanceadores de carga, detalhando seu funcionamento e classificações, incluindo abordagens em nível de aplicação e em nível de rede, tanto em contextos tradicionais quanto no âmbito das Redes Definidas por Software (SDNs). Em seguida, o foco se voltará aos servidores de distribuição de conteúdo e às Redes de Distribuição de Conteúdo (CDNs), explorando as principais arquiteturas empregadas em seus servidores de cache e os modelos de persistência de dados utilizados. Isso incluirá uma análise do funcionamento básico do cache nesses sistemas, um componente crucial para a eficiência da entrega de conteúdo.

Posteriormente, serão investigadas as estratégias e ferramentas para o monitoramento de memória, tanto principal (cache) quanto secundária (disco), em servidores de distribuição de conteúdo. Este estudo se aprofundará nas técnicas para a transmissão contínua dos dados de monitoramento de memória, seja via aplicação ou via rede. A compreensão desses mecanismos é vital, pois o uso de memória é um indicador-chave do desempenho em servidores cujo papel principal é a recuperação e entrega de dados.

A consolidação desses conhecimentos permitirá o estabelecimento conceitual de um algoritmo de escolha de servidores pelo balanceador de carga, fundamentado nos dados de monitoramento de memória, alinhando-se ao objetivo geral de elaborar, implementar e analisar tal algoritmo.

2.1 BALANCEADORES DE CARGA

Os balanceadores de carga são componentes cruciais na arquitetura de sistemas distribuídos contemporâneos, incluindo ambientes de computação em nuvem e, por extensão, os Sistemas de Distribuição de Conteúdo (CDNs) que são o foco deste trabalho. A função primordial de um balanceador de carga é distribuir de maneira eficiente e inteligente as cargas de trabalho – como requisições de usuários, tráfego de rede ou tarefas de processamento – entre múltiplos recursos computacionais, tipicamente servidores.

A escalabilidade horizontal, caracterizada pela adição de mais computadores a um sistema, tem-se consolidado como uma estratégia preferencial frente à escalabilidade vertical (o aumento de recursos em um computador existente), por sua maior flexibilidade e otimização de custos (Jane, 2025). Como consequência direta dessa abordagem distribuída, os balanceadores de carga assumem um papel crítico, sendo responsáveis por gerenciar a redundância de servidores e direcionar as requisições de forma inteligente, assegurando assim uma utilização eficaz dos recursos.

Esta distribuição estratégica é vital para otimizar a utilização dos recursos disponíveis,

maximizar a vazão (*throughput*) de requisições atendidas, minimizar o tempo de resposta para o usuário final e, fundamentalmente, evitar a sobrecarga em qualquer servidor individual. Ao prevenir que um único servidor se torne um ponto de estrangulamento, o balanceamento de carga não apenas aprimora o desempenho geral e a capacidade de escalonamento do sistema, mas também eleva significativamente sua confiabilidade e disponibilidade. Isso ocorre, por exemplo, pela capacidade de detectar falhas em servidores e redirecionar o tráfego para os servidores operantes, garantindo a continuidade do serviço.

A eficácia de um sistema distribuído moderno, portanto, está intrinsecamente ligada à sofisticação e adequação de suas estratégias de balanceamento de carga. Tendo em vista que os balanceadores de carga podem ser concebidos em duas principais categorias - em nível de aplicação e em nível de rede (Pandian; Fernando; Haoxiang, 2022) - nas subseções seguintes serão explorados esses conceitos, diferentes tipos e abordagens de balanceadores de carga, fornecendo a base necessária para a discussão de sua otimização no contexto específico da distribuição de conteúdo baseada em monitoramento de memória.

2.1.1 Balanceadores de carga em nível de aplicação

Os balanceadores de carga em nível de aplicação, também conhecidos como balanceadores de Camada 7 (L7) do modelo OSI ou camada de Aplicação no modelo TCP/IP, operam inspecionando os dados da própria aplicação. Isso significa que eles tomam decisões de roteamento com base em informações contidas no tráfego da aplicação, como cabeçalhos HTTP/HTTPS, cookies, tipo de conteúdo (imagens, vídeo, texto) ou qualquer outro dado específico da mensagem.

Diferentemente dos balanceadores de nível de rede, os balanceadores L7 analisam o conteúdo do tráfego, o que permite uma distribuição de carga muito mais granular e inteligente. Por exemplo, um balanceador de carga L7 pode direcionar requisições para servidores diferentes com base na URL solicitada (roteamento baseado em caminho), no tipo de navegador do usuário, ou na linguagem preferida indicada no cabeçalho da requisição. Uma requisição para uma URL de um serviço de arquivos pode ser enviada para um *pool* de servidores otimizado para servir imagens, enquanto requisições para uma *API* podem ser encaminhadas dinamicamente para um *pool* de servidores de aplicação.

Quanto às suas vantagens, os balanceadores L7 permitem decisões de roteamento mais inteligentes e flexíveis, o que resulta em maior eficiência na distribuição de carga, especialmente para aplicações web complexas. Adicionalmente, eles oferecem a capacidade de descarregar tarefas computacionalmente intensivas, como a terminação SSL, dos servidores, otimizando o uso de recursos destes.

Por outro lado, os balanceadores de nível de aplicação geralmente consomem mais recursos computacionais (CPU e memória) em comparação com os balanceadores de nível de rede, devido à necessidade de inspecionar e processar o conteúdo detalhado dos pacotes. Este processamento adicional também pode introduzir uma latência ligeiramente maior no encaminhamento das requisições.

Alguns exemplos de balanceadores de carga que atuam na camada de aplicação são:

1. **NGINX:** Amplamente utilizado como servidor web, proxy reverso e balanceador de carga L7, capaz de rotear tráfego HTTP/HTTPS com base em URLs, cabeçalhos, etc.
2. **HAProxy:** Outra solução popular de código aberto que pode operar tanto em L4 quanto em L7, oferecendo funcionalidades avançadas de balanceamento HTTP.
3. **AWS Application Load Balancer (ALB):** Um serviço gerenciado da Amazon Web Services que opera na camada 7, permitindo roteamento avançado de requisições HTTP/HTTPS.
4. **Azure Application Gateway:** O balanceador de carga L7 da Microsoft Azure, oferecendo funcionalidades como afinidade de sessão baseada em cookies, terminação SSL e *firewall* de aplicação web (WAF).

2.1.2 Balanceadores de carga em nível de rede

A introdução das Redes Definidas por Software (SDNs) trouxe uma nova perspectiva no contexto de balanceamento de carga, permitindo sua realização a nível de rede sem depender de uma aplicação. A arquitetura SDN, ao desacoplar o plano de controle do plano de dados, trouxe uma **gestão centralizada e programável do encaminhamento de tráfego**, o que é particularmente vantajoso para as estratégias de balanceamento de carga (Belgaum et al., 2020). O controlador SDN, possuindo uma visão global e em tempo real do estado da rede – incluindo topologia, carga dos enlaces e até mesmo estado dos servidores –, pode realizar a maior parte das funções que uma aplicação gerenciadora de balanceamento de carga normalmente pode realizar (Hamdan et al., 2021). Dessa forma, pode-se conceber que balanceadores de carga de nível de aplicação possuem como principal vantagem a flexibilidade em detrimento do desempenho, enquanto balanceadores de carga de nível de rede possuem como principal vantagem o desempenho mas trazendo como desvantagem uma complexidade e configurabilidade elevadas.

Neste paradigma, a lógica do balanceador de carga pode ser implementada como uma aplicação que reside no topo do controlador SDN. Esta aplicação analisa as informações da rede coletadas pelo controlador e, com base em algoritmos de balanceamento predefinidos ou adaptativos, instrui os dispositivos de encaminhamento (como switches OpenFlow) sobre como distribuir os fluxos de tráfego entre os servidores disponíveis. Por exemplo, o controlador pode instalar regras nos switches para que o tráfego destinado a um determinado endereço IP virtual (VIP) seja distribuído entre um conjunto de endereços IP reais de servidores, utilizando métricas como a menor carga do servidor, menor latência ou maior largura de banda disponível. Esta capacidade de **programação dinâmica das tabelas de fluxo** nos switches permite a implementação de políticas de balanceamento de carga granulares e a rápida adaptação a mudanças na rede ou na carga dos servidores, sem a necessidade de reconfiguração manual de dispositivos individuais.

Uma das principais vantagens do balanceamento de carga em SDNs é a **otimização do uso dos recursos da rede e dos servidores**. Com a visibilidade global, o controlador SDN pode evitar gargalos e subutilização de recursos de forma mais eficaz (Blial; Mamoun; Benaini, 2016). Além disso, a centralização da inteligência de balanceamento no controlador pode **simplificar a infraestrutura de rede**, potencialmente reduzindo a necessidade de dispositivos de balanceamento de carga dedicados e caros. A programabilidade também facilita a implementação de algoritmos de balanceamento mais sofisticados que podem considerar múltiplos fatores simultaneamente, realizando a gestão inteligente dos fluxos em nível de rede com base em metadados e estatísticas da rede. A capacidade de resposta a eventos da rede, como falhas de servidores ou congestão de enlaces, é também aprimorada, permitindo um redirecionamento de tráfego mais ágil e eficiente para manter a disponibilidade do serviço.

2.2 MPEG-DASH

O *streaming* via HTTP se consolidou como prática tecnológica mais aplicada para entrega de conteúdo multimídia graças à popularização do consumo de conteúdo na web, à consequente abundância de plataformas web e às conexões de banda larga (Begen; Akgul; Baugher, 2011). Seguindo essa tendência, um novo padrão - *Dynamic Adaptive Streaming over HTTP* (ou simplesmente DASH) - foi desenvolvido de modo a fornecer dinamismo e flexibilidade em diversas situações distintas de conexões de rede. O padrão contrasta com protocolos mais antigos como o RTP (*Real-Time Transport Protocol*), que frequentemente enfrentam problemas com *firewalls* e exigem que os servidores mantenham um estado de sessão para cada cliente. O HTTP oferece uma abordagem sem estado (*stateless*) mais escalável e compatível com a infraestrutura existente (Sodagar, 2011). Aproveitando essa base, foi desenvolvido o padrão *Dynamic Adaptive Streaming over HTTP* (MPEG-DASH), com o objetivo de criar uma solução interoperável para o *streaming* adaptativo, unificando um mercado anteriormente fragmentado por soluções proprietárias (Sodagar, 2011).

A arquitetura do MPEG-DASH é fundamentalmente orientada ao cliente. O servidor de conteúdo tem a função de preparar e disponibilizar o material, mas toda a inteligência da sessão de *streaming* reside no cliente (Stockhammer, 2011). Isso significa que o servidor não precisa gerenciar o estado da conexão, permitindo que servidores HTTP e redes de distribuição de conteúdo (CDNs) convencionais sejam utilizados para a entrega em larga escala de forma eficiente (Stockhammer, 2011). A operação do padrão se baseia em dois componentes principais:

1. **Media Presentation Description (MPD):** É um arquivo de manifesto, geralmente em formato XML, que descreve a estrutura e as diferentes versões do conteúdo multimídia disponíveis (Sodagar, 2011). O MPD organiza o conteúdo hierarquicamente em Períodos (*Periods*), que representam intervalos de tempo; Conjuntos de Adaptação (*Adaptation Sets*), que agrupam alternativas de um mesmo componente de mídia (como diferentes idiomas de áudio ou ângulos de câmera); e Representações (*Representations*), que são as

diferentes versões de um mesmo fluxo, codificadas com distintas taxas de bits, resoluções ou qualidades (Sodagar, 2011). O MPD contém os metadados essenciais, incluindo as URLs para acessar cada parte do conteúdo.

2. **Segmentos:** O conteúdo de mídia em si é dividido em pequenos blocos sequenciais chamados segmentos (Sodagar, 2011). Cada representação possui sua própria sequência de segmentos, que são requisitados individualmente pelo cliente através de requisições HTTP GET padrão (Stockhammer, 2011). Por serem arquivos independentes acessados via HTTP, os segmentos são facilmente armazenáveis em cache por proxies e CDNs, o que é um dos pilares da escalabilidade do DASH.

O fluxo de trabalho típico de uma sessão DASH começa com o cliente requisitando e analisando o arquivo MPD (Sodagar, 2011). Com base nas informações do manifesto, nas condições atuais da rede (como a largura de banda estimada), nas capacidades do dispositivo (como resolução de tela) e nas preferências do usuário, a lógica de controle do cliente (conhecida como *decision engine* ou *control heuristics*) seleciona a representação mais apropriada para iniciar a reprodução (Begen; Akgul; Baugher, 2011). É importante notar que essa lógica de decisão e adaptação não é padronizada pelo MPEG-DASH, o que confere flexibilidade aos desenvolvedores de clientes (Sodagar, 2011). Após o início da reprodução, o cliente continua a requisitar os segmentos sequencialmente, monitorando o desempenho da rede e, se necessário, trocando para uma representação de maior ou menor qualidade para garantir uma reprodução contínua e com a melhor Experiência do Usuário (QoE) possível.

2.2.1 *Content Steering*

O *Content Steering* (direcionamento de conteúdo) é um mecanismo que estende a capacidade de adaptação do cliente para além da simples seleção da qualidade do fluxo de mídia. Em vez de apenas decidir *o que* buscar (qual representação), o *Content Steering* permite ao cliente decidir *de onde* buscar o conteúdo, ou seja, de qual servidor ou CDN obter os segmentos (Sodagar, 2011).

O padrão MPEG-DASH facilita essa funcionalidade diretamente no *Media Presentation Description* (MPD) através da especificação de múltiplas URLs base (*BaseURL*). O MPD pode listar vários endereços para o mesmo conjunto de segmentos, onde cada URL pode apontar para um servidor espelho, um datacenter geograficamente distinto ou até mesmo uma CDN completamente diferente (Sodagar, 2011).

A responsabilidade de escolher qual URL base utilizar recai inteiramente sobre a lógica de controle do cliente, pois o padrão apenas fornece o mecanismo, mas não dita a política de seleção (Sodagar, 2011). Um cliente pode implementar estratégias de direcionamento para:

- **Otimizar o desempenho:** O cliente pode medir a latência ou a vazão para cada uma das URLs base disponíveis e selecionar aquela que oferece o melhor desempenho em um

dado momento.

- **Aumentar a confiabilidade:** Caso a requisição para um segmento a partir de uma URL base falhe ou se torne excessivamente lenta, o cliente pode dinamicamente tentar buscar o mesmo segmento (ou os segmentos subsequentes) a partir de uma URL base alternativa, criando um mecanismo de *failover* no nível da aplicação.
- **Balancear a carga:** Em cenários mais avançados, o cliente poderia distribuir suas requisições entre múltiplas fontes de conteúdo para maximizar a largura de banda disponível e evitar sobrecarregar um único servidor ou rota de rede (Sodagar, 2011).

Dessa forma, o *Content Steering* transforma o cliente DASH em um participante ativo no balanceamento de carga e na otimização da entrega, tornando o padrão altamente resiliente e adaptável a implantações de grande escala que envolvem múltiplas CDNs e infraestruturas de servidores complexas.

2.3 SERVIDORES DE DISTRIBUIÇÃO DE CONTEÚDO E CDNS

As Redes de Entrega de Conteúdo (popularmente conhecidas como *Content Delivery Networks* ou CDNs) são uma parte crítica da infraestrutura da Internet, criada para distribuir conteúdo de forma rápida, confiável e eficiente. Elas atingem esse objetivo por meio da distribuição redundante de conteúdo em vários servidores geograficamente dispersos. Quando um usuário realiza uma requisição para um conteúdo por meio de um serviço *web*, a requisição é encaminhada dinamicamente para o servidor mais próximo do usuário em vez de ser enviada para o servidor de origem do conteúdo. Essa é uma prática simples que reduz a latência, minimiza a perda de pacotes (por diminuir o número de saltos) e remove a sobrecarga de servidores que possuem o conteúdo original, resultando em uma QoE (*Quality of Experience*) mais elevada.

É interessante notar também que as Redes de Entrega de Conteúdo, apesar de serem conjuntos de dispositivos computacionais interconectados assim como as redes de computadores tradicionais, não são necessariamente um subconjunto dessas redes no contexto conceitual. Em vez disso, são sistemas distribuídos de computadores, geralmente administrados a nível de aplicação por uma organização privada. Em outras palavras, são grupos de computadores que se comunicam de forma padronizada para cumprir com o objetivo de entregar conteúdo para os usuários finais, mas que não estão no mesmo nível de abstração conceitual das redes de computadores. Dessa forma, as CDNs são, na verdade, arquiteturas de aplicação altamente complexas e flexíveis e que, por extensão, estão localizadas no topo da pilha de abstração de camadas de rede - a camada de aplicação.

Nesse sentido, considerando que o princípio central das CDNs é aumentar a taxa de transmissão e diminuir a latência por meio da aproximação física do conteúdo aos usuários, os seus mecanismos-chave são:

1. **Arquitetura:** Predominantemente, as CDNs se organizam em duas camadas principais de servidores: **Os servidores de origem** que detêm o conteúdo original e os servidores de borda que possuem cópias redundantes dos conteúdos e são posicionados estrategicamente mais perto dos usuários finais. Esses servidores de borda são organizados em PoPs (*Points of Presence*, Data Centers organizados com o intuito de dispor servidores de borda para as CDNs.
2. **Cache:** As CDNs possuem vários servidores de borda localizados em regiões geográficas distribuídas. Quando um usuário realiza uma requisição para o conteúdo em questão, a CDN responde com o conteúdo do servidor localizado no PoP mais próximo, significativamente reduzindo a distância física sobre qual a requisição precisa trafegar. Esse funcionamento básico reduz a latência e aumenta a taxa de transmissão de dados, ainda sendo efetivo para a Internet sob o ponto de vista de ecossistema, uma vez que há menos requisições geograficamente dispersas evitando sobrecarregar a rede global.
3. **Roteamento:** As CDNs se beneficiam de várias técnicas para determinar o melhor PoP para lidar com uma requisição do usuário. Uma das técnicas para realizar esse processo é a utilização de **roteamento baseado em DNS**, no qual o sistema DNS da CDN direciona o usuário para o endereço de IP do servidor mais apropriado baseado na localização geográfica.
4. **Balanceamento de carga:** As redes de entrega de conteúdo distribuem requisições dos usuários para evitar que quaisquer servidores de um PoP se sobrecarreguem. Conforme mencionado, o principal fator para distribuição de requisições entre servidores é a localização geográfica. No entanto, outras técnicas mais avançadas também são empregadas, como *anycast* (Preston, 2021), na qual os servidores de borda anunciam possuir o mesmo endereço de IP através de um roteador BGP e, naturalmente, o servidor geograficamente mais próximo da requisição efetivamente recebe e a processa.

Para os **servidores de borda** das CDNs, que estão dispostos em larga escala, se configurados apenas com as tarefas de entrega de conteúdo estático pré-processado (como é o caso do *MPEG-DASH*), é correto dizer que eles teriam menos tarefas intensivas de processo específicas de conteúdo no momento da entrega em comparação com, por exemplo, a transcodificação de vídeo instantânea ou a geração de conteúdo dinâmico durante o tratamento de cada requisição. Nesse sentido, faz sentido conceber que os recursos computacionais mais utilizados nesse caso são os recursos de armazenamento - tanto em memória principal quanto em memória secundária.

2.3.1 Funcionamento de cache em redes de entrega de conteúdo

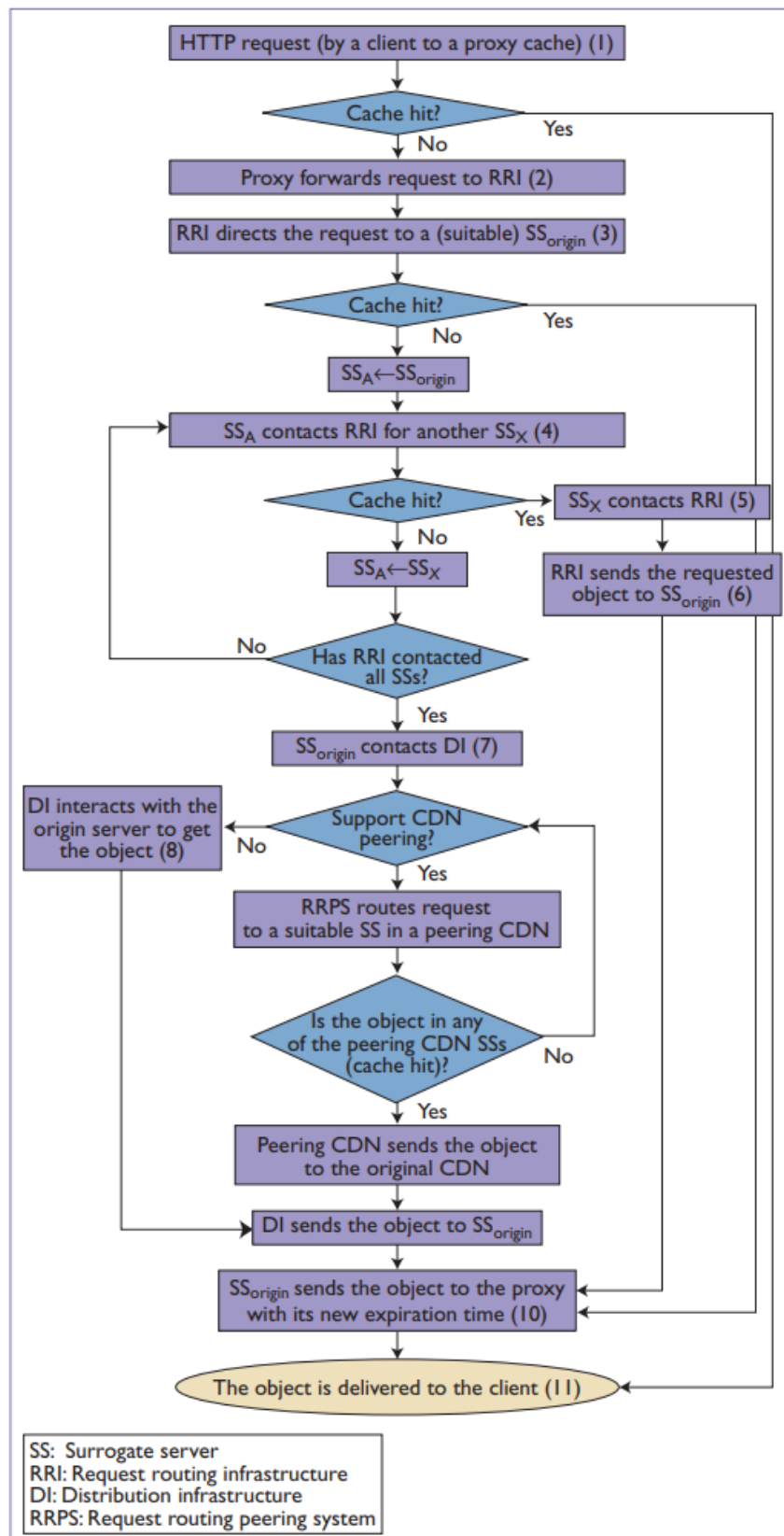
Na complexidade intrínseca às CDNs, se encontram múltiplas camadas de *caching* de modo a cumprir com o propósito de evitar tráfegos em amplas distâncias. Conforme a Figura 1,

é possível conceber pelo menos três níveis nas arquiteturas tradicionais de CDNs (Vakali; Pallis, 2003):

1. **Acesso direto ao Proxy:** Na primeira iteração de uma requisição que passa por uma CDN, é realizada uma verificação nos servidores de proxy dos provedores de rede (ISPs). Os **servidores de proxy** são basicamente computadores (físicos ou virtuais) dispostos diretamente nas infraestruturas dos provedores de serviço, mantidos para armazenar conteúdo frequentemente acessado pelos usuários e até mesmo evitar sobrecarga nos servidores de borda das CDNs.
2. **Acesso iterativo aos servidores de borda da CDN:** Em segundo plano, são realizadas tentativas sucessivas de acesso ao conteúdo requisitado através dos servidores de borda da CDN, um por vez. Ao encontrar o conteúdo em questão, ele pode ser armazenado em cache nos servidores nos quais o conteúdo não foi encontrado.
3. **Peering de CDNs:** Em algumas CDNs específicas, maiores e mais complexas, é possível que seja realizada uma configuração de colaboração (*peering*) que faz com que CDNs possam buscar por conteúdo em outras CDNs, aumentando ainda mais a eficiência.

É válido conceber que o funcionamento geral de uma CDN consiste na disposição e organização de servidores (físicos e virtuais) de modo a minimizar a distância percorrida por pacotes de requisições na Internet. Além disso, sabe-se que a ferramenta tecnológica mais utilizada para escolher dinamicamente o servidor conforme o conteúdo requisitado e distâncias geográficas é o *roteamento dinâmico de DNS*, considerada a técnica mais popular no contexto das CDNs (Wang; Huang; Rose, 2018).

Figura 1 – Fluxo detalhado das múltiplas camadas de *caching* em CDNs.



Fonte: Adaptado de (Vakali; Pallis, 2003)

2.4 MONITORAMENTO DE MEMÓRIA

Com a emergente necessidade de modernizar e escalar sistemas nos mais diversos setores da indústria, por consequência, emergiu também, no contexto tecnológico, a necessidade da disponibilização de recursos computacionais sob demanda de modo mais flexível possível. Esse fenômeno resultou no paradigma de disponibilização de recursos computacionais de forma mais abstrata e encapsulada, conhecido como Computação em Nuvem (*Cloud Computing*) (Boss et al., 2007). Junto com a aplicação do novo paradigma, também se fez necessária a evolução sistemática dos processos de monitoramento de recursos computacionais e de rede, como vazão de dados, latência, processamento da CPU, uso de memória principal e secundária, dentre outros.

2.4.1 APIs fornecidas pelos principais sistemas operacionais

A principal e mais utilizada maneira de se obter a métrica do consumo de memória é por meio de APIs disponibilizadas pelos Sistemas Operacionais. A maioria dos *Kernels*, durante o gerenciamento de memória e de disco relacionados aos processos gerenciados, possui instruções para armazenar e recuperar informações, atualizadas constantemente, a respeito do consumo de memória e de disco. Embora a disposição dessas informações seja diferente de sistema para sistema, a maior parte deles disponibiliza uma API para que aplicações no espaço do usuário (*user-space*) consigam obter essa informação.

Para o sistema operacional Linux, por exemplo, a interface primária para monitoramento é o diretório virtual `/proc/meminfo`. Através deste diretório, o SO fornece uma análise detalhada do uso da RAM (*Random Access Memory* - ou Memória Principal) (The Linux Kernel Developers, 2020). Além disso, o SO fornece informações detalhadas sobre estatísticas de I/O para cada bloco no disco (memória secundária) por meio do diretório `/proc/diskstats` (The Linux Kernel Developers, 2011). Outros sistemas operacionais possuem seus próprios métodos e APIs para disponibilizar essas informações. Enquanto o *macOS* utiliza subsistemas (Apple Inc., 2013) (Apple Inc., 2012), o Windows possui uma API bem definida baseada em requisições denominada *Performance Counters*, que sistematicamente disponibiliza essas informações (Microsoft Corporation, 2023b) (Microsoft Corporation, 2023a).

2.4.2 Aplicações

Com base no princípio anterior, existem várias soluções à nível de aplicação que reференçiam o sistema operacional de forma dinâmica para dispor, transportar e processar essas informações de monitoramento de forma sistemática. Alguns exemplos são: o sistema *Tivoli® Monitoring* da IBM (Boss et al., 2007), o clássico *Task Manager* do Windows, as soluções de linha de comando `top`, `htop` e `free` do Linux, dentre outras aplicações que podem funcionar tanto de forma isolada quanto distribuída.

2.5 TRANSMISSÃO CONTÍNUA DE DADOS

Em sistemas distribuídos compostos por múltiplas máquinas (virtuais ou físicas), existem várias técnicas empregadas nas redes de computadores para transmitir dados de forma contínua entre dispositivos. Em diferentes níveis de abstração, granularidade e complexidade, elas podem ser usadas dentro do contexto de uma aplicação para, efetivamente, com os dados sobre consumo de memória fornecidos pelo sistema operacional, transmitir essa informação de modo periódico e contínuo entre sistemas.

Uma das estratégias cuja implementação tem intrínseca simplicidade é a criação de APIs REST (*Representational State Transfer*), que consiste em um estilo arquitetural para sistemas distribuídos baseado em um conjunto de restrições, como a comunicação sem estado (*stateless*). Utilizando os verbos do protocolo HTTP (como GET, POST, PUT, DELETE), os sistemas clientes podem requisitar ou enviar dados periodicamente para um servidor. Cada requisição é independente e deve conter toda a informação necessária, tornando a arquitetura simples, escalável e fácil de implementar com as tecnologias web existentes (Mozilla Developer Network, 2024).

Uma alternativa mais formal e robusta é o SOAP (*Simple Object Access Protocol*), um protocolo baseado em XML com uma especificação rígida para a troca de mensagens. Diferente do REST, o SOAP define um envelope padronizado para as mensagens e utiliza um arquivo WSDL (*Web Services Description Language*) para descrever o contrato do serviço. Embora seja mais verboso e complexo que o REST, oferece funcionalidades avançadas de segurança e transações (WS-Security), sendo frequentemente utilizado em ambientes corporativos e sistemas legados (Gudgin et al., 2007).

Para cenários que exigem baixa latência e comunicação em tempo real, o protocolo WebSockets é uma excelente opção. Ele estabelece uma única conexão TCP de longa duração entre o cliente e o servidor, permitindo a troca de dados bidirecional a qualquer momento. Isso elimina o overhead de múltiplas requisições HTTP e possibilita que o servidor envie dados de forma proativa (*push*) para o cliente, tornando-o ideal para a transmissão contínua de dados de monitoramento com mínima demora (Fette; Melnikov, 2011).

Em ambientes de microsserviços onde o desempenho é crítico, o gRPC (*Google Remote Procedure Call*) se destaca. Utilizando HTTP/2 para transporte e *Protocol Buffers* para serialização de dados, o gRPC oferece uma comunicação binária de alta performance. Ele suporta streaming bidirecional, permitindo que cliente e servidor enviem um fluxo contínuo de mensagens sobre uma única conexão, sendo significativamente mais eficiente em termos de latência e uso de banda em comparação com APIs baseadas em JSON/REST (gRPC, 2024).

O modelo de publicação/subscrição (*publish/subscribe*), implementado por *message brokers*, oferece uma arquitetura desacoplada. Protocolos como MQTT (*Message Queuing Telemetry Transport*), extremamente leve e projetado para dispositivos com recursos limitados, ou AMQP (*Advanced Message Queuing Protocol*), mais robusto e com garantias de entrega,

permitem que um sistema publique dados em um "tópico" central. Outros sistemas, inscritos nesse tópico, recebem os dados instantaneamente. Essa abordagem é altamente escalável e resiliente, ideal para sistemas distribuídos em larga escala (OASIS, 2012) (OASIS, 2019).

Uma abordagem mais simples, embora com maior latência, é o *polling* de arquivos. Neste método, um sistema gera periodicamente um arquivo contendo os dados de monitoramento e o deposita em um local compartilhado, como um servidor FTP/SFTP. O sistema receptor, por sua vez, varre esse local em intervalos regulares para processar novos arquivos. Embora não seja uma solução em tempo real, sua simplicidade e robustez a tornam viável para processos de *batch* onde a instantaneidade não é um requisito (Postel; Reynolds, 1985).

Finalmente, para obter controle máximo e o menor *overhead* de protocolo possível, pode-se trabalhar diretamente na camada de transporte utilizando *sockets*. Com *sockets* TCP, estabelece-se uma conexão confiável e orientada à conexão, garantindo a entrega ordenada dos pacotes. Já com *sockets* UDP, enviam-se datagramas de forma não confiável e sem conexão, priorizando a velocidade sobre a garantia de entrega. Ambas as abordagens exigem a implementação manual do protocolo de aplicação para serialização e enquadramento de dados, representando a solução de mais baixo nível e maior complexidade (Postel, 1981) (Postel, 1980).

2.6 INTERPRETAÇÃO DE DADOS OBSOLETOS

No contexto de balanceamento de carga, é fato que a estratégia de direcionamento da carga para a máquina com menos sobrecarga pode desempenhar mal se a informação não é suficientemente próxima do tempo real (Eager; Lazowska; Zahorjan, 1986; Mirchandaney; Towsley; Stankovic, 1989; Mitzenmacher, 2000; Shivaratri; Krueger; Singhal, 1993). Nesse sentido, sistemas de balanceamento de carga que direcionam a carga para servidores pouco sobrecarregados e que não tratam devidamente a obsolescência dessas informações de sobrecarga podem produzir o efeito contrário ao desejado (que é evitar a sobrecarga em uma ou mais máquinas), efetivamente sobrecarregando as máquinas que processam requisições.

A falta de tratamento devido a dados obsoletos pode causar um fenômeno no qual máquinas que aparentam estar subutilizadas rapidamente se tornam sobrecarregadas, uma vez que, durante a defasagem da propagação da informação, todos os clientes enviam suas requisições para as mesmas máquinas, que por sua vez ficam sobrecarregadas até que uma nova informação de carga seja propagada pelo sistema. Para combater esse problema, algumas abordagens comuns incluem o uso de algoritmos de seleção aleatória ou *Round-Robin*, que ignoram completamente os dados de carga, ou o uso de um subconjunto aleatório de servidores para a tomada de decisão.

Por outro lado, se for imprescindível que os dados de sobrecarga de um ou mais recursos computacionais dos servidores gerenciados sejam considerados como variáveis utilizadas pela estratégia de balanceamento de carga, existem algumas estratégias para resolver o problema da obsolescência dos dados. O trabalho "*Interpreting Stale Load Information*" desenvolve estratégias que interpretam a informação de carga com base em sua idade em vez de arriscar um desempenho

extremamente baixo ou ignorar a oportunidade de usar essa informação para otimizar o sistema (Dahlin, 2000).

2.6.1 Algoritmos de Interpretação de Carga (Load Interpretation - LI)

A ideia central do trabalho é utilizar não apenas a última informação de carga reportada por um servidor (L_i), mas também a idade dessa informação (T) e uma estimativa da taxa de chegada de novos trabalhos (λ). Com base nesses parâmetros, os algoritmos ajustam a distribuição de requisições.

O artigo propõe e avalia principalmente dois algoritmos baseados nesse princípio:

- **Basic Load Interpretation (Basic LI):** Este algoritmo calcula a fração de requisições que deve ser enviada para cada servidor com o objetivo de equalizar o tamanho das filas (carga) em todos os servidores ao final de um determinado período de tempo (fase). A probabilidade (P_i) de uma requisição ser enviada ao servidor i é calculada para que, ao final da fase, a soma dos trabalhos existentes e dos que chegaram seja distribuída de forma equitativa.
- **Aggressive Load Interpretation (Aggressive LI):** Considerando que o algoritmo básico pode levar muito tempo para equilibrar a carga em fases longas, a versão agressiva busca otimizar esse processo. Ela subdivide a fase em intervalos e utiliza a primeira parte para nivelar a carga de todos os servidores, distribuindo as requisições restantes de maneira uniforme.

2.6.2 Desempenho e Conclusões

Por meio de simulações, o estudo demonstra que os algoritmos de LI são eficazes em uma vasta gama de condições. Os resultados sugerem que, ao interpretar corretamente a informação de carga, os sistemas podem:

- Igualar o desempenho dos algoritmos mais agressivos quando a informação de carga é recente.
- Superar os melhores algoritmos alternativos em até 60% quando a informação é moderadamente obsoleta.
- Apresentar um desempenho significativamente superior à distribuição aleatória quando a informação é ainda mais antiga.
- Evitar comportamento patológico mesmo quando a informação é extremamente obsoleta.

Além disso, os algoritmos de LI mostram-se robustos e mantêm um bom desempenho mesmo com padrões de chegada de trabalhos intermitentes (*bursty*) e com tamanhos de trabalho muito variáveis. A abordagem resolve o paradoxo no qual o uso de mais informação pode resultar

em um desempenho pior, pois os algoritmos de LI ajustam seu comportamento, tornando-se mais conservadores à medida que a informação se torna mais obsoleta.

2.7 FILAS DE MENSAGENS PARA COMUNICAÇÃO EM SISTEMAS DISTRIBUÍDOS

No paradigma atual de sistemas distribuídos, as filas de mensagens são um elemento arquitetônico essencial, projetadas para mediar a comunicação e distribuir cargas de trabalho de forma eficiente. Em vez de utilizar chamadas de API diretas e síncronas — que geram acoplamento forte entre serviços e aumentam a latência —, as filas de mensagens empregam um modelo de comunicação assíncrona. Nesse modelo, diferentes componentes do sistema podem interagir sem depender diretamente da disponibilidade um do outro, pois as mensagens são armazenadas em um intermediário até que o serviço de destino esteja pronto para processá-las (Thallapally, 2025).

A arquitetura de uma fila de mensagens é composta por três componentes chave. Os Produtores (*producers*) são os serviços ou aplicações que criam e enviam as mensagens, que podem conter desde simples requisições a eventos com dados complexos (*payloads*). O Broker de Mensagens (*Message Broker*) atua como o *middleware* central, gerenciando o armazenamento, o roteamento e a entrega das mensagens. Finalmente, os Consumidores (*Consumers*) são os serviços que recebem as mensagens da fila para processá-las conforme necessário. Essa separação entre quem envia e quem recebe é o que viabiliza os principais benefícios da tecnologia.

Uma das vantagens mais significativas das filas de mensagens é o desacoplamento de serviços, que permite que produtores e consumidores operem de forma independente. O *broker* de mensagens funciona como um intermediário, eliminando as dependências diretas que dificultam a escalabilidade e a manutenção em modelos de comunicação síncrona. Esse baixo acoplamento permite que componentes individuais sejam atualizados, mantidos ou escalados sem impactar o sistema como um todo. Consequentemente, as filas melhoram drasticamente a escalabilidade horizontal, pois permitem que mensagens sejam distribuídas para múltiplos consumidores que as processam em paralelo, mantendo a eficiência do sistema mesmo sob alta demanda (Thallapally, 2025).

Além da escalabilidade, as filas de mensagens são cruciais para garantir a tolerância a falhas e a confiabilidade em aplicações distribuídas. Muitos sistemas de fila utilizam armazenamento persistente, gravando as mensagens em disco até que sejam processadas com sucesso, o que as protege contra perdas em caso de falhas ou reinicializações do sistema. Para lidar com mensagens que falham repetidamente no processamento, é comum o uso de *Dead Letter Queues* (DLQs), que são filas específicas para armazenar essas mensagens problemáticas para análise posterior, evitando sua perda e ajudando a identificar falhas recorrentes no sistema.

Devido a essas características, as filas de mensagens são amplamente utilizadas em diversas indústrias e aplicações modernas. Em plataformas de e-commerce, elas gerenciam o processamento de pedidos, atualizações de inventário e notificações a clientes. No setor

financeiro, são usadas para processamento seguro de transações e detecção de fraudes. Sistemas de Internet das Coisas (IoT) dependem delas para agregar dados de sensores e processar eventos em tempo real, enquanto aplicações de mídia social as utilizam para notificações assíncronas e distribuição de conteúdo. Em suma, as filas de mensagens são um pilar para a construção de sistemas distribuídos resilientes, escaláveis e eficientes.

2.8 VIRTUALIZAÇÃO DE REDES PARA SIMULAÇÃO DE CENÁRIOS

A virtualização de redes surgiu como uma abordagem para flexibilização da adição de novas tecnologias na arquitetura da Internet, extremamente necessária em um cenário no qual a introdução de novas tecnologias radicalmente diferentes se tornou praticamente impossível devido à necessidade de consenso entre múltiplos *stakeholders* com objetivos conflitantes. Esse conceito propõe permitir a coexistência de múltiplas arquiteturas de rede heterogêneas sobre uma infraestrutura física compartilhada, promovendo assim a inovação e o desenvolvimento de aplicações diversificadas (Chowdhury; Boutaba, 2010).

Uma rede virtual é essencialmente uma coleção de nós e links virtuais, representando um subconjunto dos recursos da rede física subjacente. A ideia fundamental é separar as responsabilidades dos tradicionais Provedores de Serviço de Internet (ISPs) em duas novas entidades: os Provedores de Infraestrutura (InPs), que gerenciam os recursos físicos, e os Provedores de Serviço (SPs), que criam redes virtuais ao agregar recursos de múltiplos InPs para oferecer serviços de ponta a ponta (Chowdhury; Boutaba, 2010). Essa separação cria um ambiente dinâmico que facilita a implantação de arquiteturas heterogêneas coexistentes, superando as limitações da Internet atual. A virtualização visa atingir objetivos de design como flexibilidade, heterogeneidade, gerenciabilidade, isolamento e programabilidade.

2.8.1 Virtualização baseada em Containers Linux

Enquanto a virtualização tradicional, por meio de Máquinas Virtuais (VMs), emula uma plataforma de hardware completa, exigindo um sistema operacional convidado (*GuestOS*) completo e um software de monitoramento (VMM ou *hypervisor*), a virtualização em nível de sistema operacional, ou baseada em *containers*, adota uma abordagem mais leve. Nesta técnica, o *kernel* do sistema operacional hospedeiro é modificado para suportar múltiplas instâncias isoladas do espaço de usuário (os *containers*). Como os programas dentro dos *containers* utilizam diretamente a API do sistema hospedeiro, o overhead de virtualização é mínimo, uma vez que não há necessidade de emulação de hardware. Essa eficiência torna os *containers*, como o LXC (Linux *Containers*), uma tecnologia chave para plataformas de emulação, permitindo que múltiplos "*netboxes*" (redes virtuais encapsuladas) executem de forma concorrente e autônoma em um único PC, com recursos computacionais privados e isolados.

Para construir redes virtuais, a abstração das interfaces de rede é fundamental. Em um ambiente de *containers*, isso é alcançado pela criação de interfaces de rede virtuais (como pares

Virtual Ethernet ou VETH) que operam como túneis. O núcleo de uma "*netbox*" é um motor de simulação (como o NS-3) executando dentro de um *container*, onde os nós da rede simulada são equipados com interfaces do tipo Ethernet. Essas interfaces são então ligadas às interfaces virtuais do *container*. Os *containers* podem ser interligados por meio de *bridges* de software que rodam no sistema hospedeiro ou conectados diretamente a placas de rede físicas (NICs), permitindo a comunicação com entidades externas ou outras *netboxes*. Essa arquitetura permite que a cascata de múltiplas *netboxes* sintetize as diferentes seções de uma rede heterogênea complexa, com cada *netbox* emulando as características de uma rede específica de forma exclusiva e isolada.

2.8.2 Simulação de Redes Definidas por Software (SDN)

O ambiente isolado e modular fornecido pela virtualização de redes é ideal para a prototipagem e simulação de arquiteturas de rede avançadas, como as Redes Definidas por Software (SDN). Ferramentas como o *Mininet*, por exemplo, demonstram a eficácia de usar virtualização leve (baseada em *namespaces*, um dos pilares dos *containers*) para prototipar rapidamente redes SDN em um único computador.

Dentro de uma *netbox*, os nós virtuais podem executar qualquer tipo de software. Isso significa que é possível instanciar roteadores programáveis, como o *Click Modular Router*, ou mesmo *switches* compatíveis com o protocolo *OpenFlow*, que é uma tecnologia fundamental para SDN. Dessa forma, pode-se emular uma topologia de rede completa, composta por *switches* e *hosts* virtuais, e gerenciá-la por meio de um controlador SDN, também executando em um *container* ou em um *host* externo. Essa capacidade permite que pesquisadores e engenheiros de rede projetem, testem e validem protocolos e serviços de SDN em um ambiente realista e controlado, com alto grau de fidelidade e a um custo baixo, antes de sua implantação em hardware físico.

3 PROPOSTA

Esta seção aborda pormenorizadamente os caminhos escolhidos para a implementação do modelo de balanceamento de carga, bem como os métodos que serão utilizados para realizar a avaliação dos resultados finais.

3.1 O ALGORITMO

O primeiro passo, crucial para a determinação do modelo de otimização de balanceadores de carga, é a definição do algoritmo de balanceamento. Basicamente, a tarefa principal de um algoritmo de balanceamento de carga é o modo como a escolha do servidor será feita em cada requisição que chega ao sistema balanceador de carga.

Em particular, o algoritmo escolhido será **probabilístico**. Isso significa que, para cada servidor que poderá ser escolhido no processo, haverá uma probabilidade intrínseca baseada em parâmetros tanto individuais quanto gerais, e a estratégia de determinação dessa probabilidade é o que definirá o algoritmo em si. De acordo com o objetivo principal do trabalho, as duas principais (mas não únicas) métricas que exercerão influência sobre a determinação da probabilidade de escolha de cada servidor são o **consumo de memória RAM** e a **Taxa de Entrada e Saída de Disco**. Essas duas métricas são essenciais e vitais dentro do contexto de servidores que distribuem conteúdo, uma vez que sua única tarefa é recuperar estes arquivos do disco e efetivamente entregá-los ao solicitante.

Em paralelo, os servidores enviarão, periodicamente, os dados de monitoramento referentes ao consumo de memória principal e à taxa de leitura de disco. A ideia é que o balanceador mantenha uma estrutura dinâmica que será atualizada de forma contínua, atualizando os parâmetros necessários e efetivamente promovendo eficiência na distribuição das requisições. O método que será usado para realizar a implementação deste conceito, bem como a definição das taxas de atualização dos dados dos servidores para o balanceador de carga, serão discutidos posteriormente.

A Equação 1 utilizada no algoritmo corresponde a uma versão adaptada do *Basic LI* (Dahlin, 2000) de modo que, dada uma requisição, a determinação da probabilidade de escolha de um servidor P_i será determinada de acordo com seus parâmetros, onde:

n : Número do total de servidores.

L_i : Carga reportada no servidor i (cálculo determinado na Equação 2).

L_{tot} : Soma de todas as cargas reportadas ($\sum L_i$).

$arrive_T$: Número esperado de requisições que chegarão para o balanceador de carga durante um período de tempo T , que corresponde ao *lifetime* (tempo de vida) médio das informações de carga de cada servidor.

P_i : Probabilidade de que uma requisição que chega seja enviada para o servidor i .

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

Note-se que no contexto do trabalho (Dahlin, 2000), "carga" é um termo genérico e abstrato. Para fins de implementação e aderência com o problema proposto por este trabalho, o parâmetro de carga (L_i e consequentemente L_{tot}), aplicado na Equação 1, será calculado previamente através da coleta periódica das métricas de consumo de memória RAM e taxa de entrada e saída de memória secundária (cujo processo será descrito posteriormente), da normalização desses parâmetros em pontos percentuais e, finalmente, da mesclagem desses parâmetros para que possam ser inseridos na fórmula. Nesse sentido, a carga de um servidor (L_i) será determinada considerando:

n : Número do total de servidores.

LM_i : Consumo de memória principal no servidor i , representando a quantidade de memória usada para transferir o conteúdo no tempo T , normalizado em pontos percentuais com relação à capacidade total de memória do servidor i .

LD_i : Carga do disco no servidor i , representando a velocidade do fluxo de busca e leitura das informações do disco no tempo T , normalizada em pontos percentuais com relação a capacidade total de leitura do disco no servidor i .

TM : Taxa de uso geral de memória, determinado por $\sum_{i=0}^n \frac{LM_i}{n}$.

TD : Taxa de consumo geral de operações de disco, determinado por $\sum_{i=0}^n \frac{LD_i}{n}$.

Sendo assim, a Equação 2 define a carga, no contexto deste trabalho, de um servidor de conteúdo.

$$L_i = CM * LM_i + CD * LD_i \quad (2)$$

Nesse contexto, CM e CD são coeficientes que multiplicam as suas respectivas cargas normalizadas. Eles são definidos de acordo com as Equações 3 e 4, respectivamente, e representam a fração normalizada de memória e disco totais, respectivamente, em relação ao total conjugado de memória e disco.

$$CM = \frac{TM}{TM + TD} \quad (3)$$

$$CD = \frac{TD}{TM + TD} \quad (4)$$

Os coeficientes CM e CD são valores entre 0 e 1 de modo que $CM + CD = 1$. Eles são determinados dessa forma para que o recurso computacional que mais esteja sendo utilizado de modo geral em todos os servidores efetivamente contribua mais para o valor da carga de cada servidor L_i , de forma diretamente proporcional à taxa de consumo de cada um dos parâmetros. Finalmente, para que o sistema efetivamente esteja apto para calcular o valor de cada servidor L_i , basta que ele obtenha os valores de LM_i e LD_i de todos os servidores. Com eles, será possível calcular primeiramente TM e TD , e, com esses valores, calcular CM e CD dinamicamente.

O algoritmo é executado a cada nova requisição que chega ao balanceador. Ele utiliza os dados de carga pré-calculados para determinar dinamicamente a probabilidade (P_i) de cada servidor ser escolhido, aplicando a Equação 1. Com as probabilidades definidas, ele emprega um método de seleção aleatória ponderada para despachar a requisição, garantindo que servidores menos sobrecarregados tenham maior chance de serem selecionados. O seguinte pseudocódigo descreve seu funcionamento.

Algorithm 1 Seleção de Servidor por Requisição

Gatilho: Chegada de uma nova requisição R .

Require:

S : Lista de servidores com métricas L_i e T_i atualizadas.

Variáveis globais: L_{tot}, n .

Ensure:

Requisição R encaminhada para um servidor S_i .

- 1: **procedure** SELECIONARSERVERIDOR(R, S, L_{tot}, n)
 - 2: $T \leftarrow \text{CalcularMediaDeRequisicoes}(S)$
 - 3: $arrive_T \leftarrow \text{CalcularChegadasEsperadas}(T)$
 - 4: $P_{list} \leftarrow []$ ▷ Lista de probabilidades
 - 5: **for all** S_i, L_i in S **do**
 - 6: $numerador \leftarrow (L_{tot} + arrive_T)/n - L_i$
 - 7: $P_i \leftarrow ((L_{tot} + arrive_T)/n - L_i)/arrive_T$ ▷ Eq. 1
 - 8: Adicionar (S_i, P_i) a P_{list}
 - 9: $s_{escolhido} \leftarrow \text{SelecaoAleatoriaComPesos}(P_{list})$
 - 10: Encaminhar($R, s_{escolhido}$)
-

Para o funcionamento do algoritmo, são empregadas duas variáveis globais: L_{tot} e n . Ambas são variáveis dinâmicas e automaticamente recalculadas sempre que o balanceador de carga recebe informações de qualquer servidor. A detecção de novos servidores ocorre de forma automática e determinística quando um evento caracteriza a identificação de um servidor que ainda não foi adicionado ao *pool* de serviços. A remoção de serviços, por sua vez, é feita de forma heurística, baseada em um valor de *threshold* que representa o tempo sem receber atualizações daquele servidor. Essa estrutura confere elasticidade ao sistema, fazendo com que haja uma adaptação dinâmica conforme as mudanças na carga e na disponibilidade dos serviços.

Dessa forma, estabelece-se um algoritmo de balanceamento completamente dinâmico que se ajustará continuamente de acordo com a condição observada dos servidores.

3.2 OS REQUISITOS TECNOLÓGICOS

A infraestrutura experimental para este trabalho será implementada em um ambiente virtualizado, contido integralmente em uma única máquina física. A escolha por esta abordagem, suportada pela tecnologia de contêineres Docker, é estratégica e visa mitigar a influência de variáveis de rede externas, como latência e instabilidade de conexão. Ao confinar toda a comunicação a uma rede virtual interna gerenciada pelo Docker, o foco da análise é direcionado exclusivamente ao impacto do monitoramento de recursos, como a memória, no desempenho e na otimização do balanceador de carga, que é o objetivo central desta pesquisa. Adicionalmente, a utilização do Docker como plataforma padrão para todos os componentes do sistema garante a padronização do ambiente de execução, eliminando problemas de dependência de sistemas operacionais e bibliotecas, o que assegura a consistência e a reprodutibilidade dos experimentos.

Neste ecossistema controlado, quatro aplicações principais interagem para formar o sistema em estudo: As três principais são o servidor de conteúdo, que será replicado de modo redundante para que o cenário emule simplificadaamente uma CDN, o cliente e o balanceador de carga. Em conjunto, ainda haverá mais um serviço auxiliar que fará interações diretamente com os servidores e com o balanceador de carga, sendo ele o serviço gerenciador de filas de mensagens. O servidor de conteúdo será responsável por fornecer segmentos de vídeo de acordo com o padrão MPEG-DASH. O balanceador de carga atuará como um intermediário, recebendo as requisições dos clientes e distribuindo-as de forma inteligente, baseado na avaliação de carga, entre os servidores disponíveis. A aplicação cliente iniciará várias requisições de modo paralelo, direcionando-as ao balanceador de carga (solicitando segmentos subsequentes conforme o MPEG-DASH) e, subsequentemente, consumindo o conteúdo de mídia retornado, fechando assim o ciclo de interação do sistema. Em paralelo, os próprios servidores, periodicamente, fornecerão seus dados de carga de memória principal e secundária de forma direta para o serviço gerenciador de filas por meio de uma publicação do modelo de comunicação indireta *publish-subscribe* (Eugster et al., 2003). O balanceador de carga, que estará diretamente consumindo eventos da fila, eventualmente consumirá esses dados de modo a realimentar os parâmetros internos utilizados para os cálculos envolvidos (ver Equações 1 e 2).

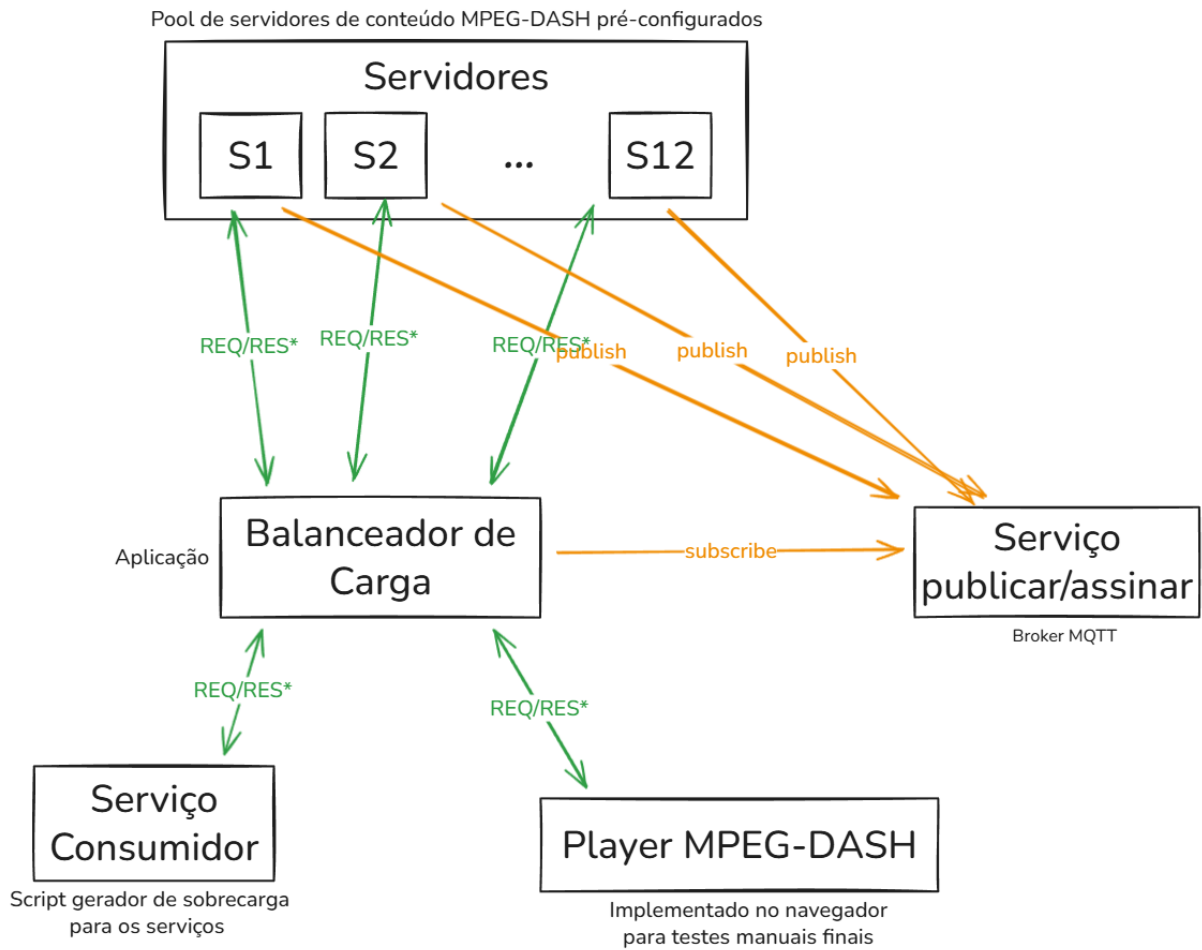
A arquitetura do sistema foi dimensionada para simular um cenário de alta demanda e é composta por uma variedade de serviços. O conjunto de recursos consiste em 12 contêineres atuando como servidores MPEG-DASH, constituindo o *pool* de nós para os quais a carga será distribuída. Haverá um contêiner dedicado para atuar como serviço gerenciador de filas de mensagens, com o propósito de fornecer os dados para a atualização dos parâmetros do balanceador de carga. Também haverá um serviço de monitoramento implementado na forma de um *script*, com acesso direto tanto aos servidores quanto ao serviço gerenciador de filas. O

sistema contará com um único balanceador de carga, que representa a peça central da investigação. Para gerar uma carga de trabalho realista e concorrente, será implementado um *script* cliente simples, projetado para gerar múltiplas requisições paralelas, simulando o comportamento de um grande número de usuários acessando o serviço simultaneamente por meio de um player MPEG-DASH. Finalmente, um simples *player* de vídeo baseado em navegador será utilizado como ferramenta de validação, cujo propósito é permitir uma avaliação qualitativa da experiência do usuário (QoE) ao reproduzir o vídeo disposto pelos servidores, mesmo sob a sobrecarga imposta pela aplicação cliente de teste.

A seleção de tecnologias para cada componente foi alinhada aos objetivos do projeto. Duas imagens *Docker* pré-existentes serão configuradas e usadas para criar os servidores MPEG-DASH e o *broker* MQTT (*Message Queuing Telemetry Transport*), respectivamente. Uma aplicação será desenvolvida e encapsulada em um contêiner na forma de um *script* Python, de modo que será responsável por simular sobrecarga nos serviços e no balanceador de carga. Por fim, o balanceador de carga, componente central desta pesquisa, será desenvolvido em *Rust*. A escolha por esta linguagem é motivada por suas características de linguagem de sistemas, que oferecem controle de baixo nível e, crucialmente, por suas garantias de segurança de memória, que eliminam classes inteiras de vulnerabilidades críticas em aplicações de rede. O design do balanceador será modular e aberto, permitindo a fácil substituição e modificação da estratégia de balanceamento, o que viabilizará futuras comparações de desempenho com algoritmos generalistas, como Round-Robin e Seleção Aleatória.

De modo abstrato, o panorama geral da arquitetura final do sistema que será criado é visualizável na Figura 2.

Figura 2 – Arquitetura geral do sistema.



* Requisição e Resposta de conteúdo em áudio/vídeo

Fonte: Elaborado pelo autor (2025)

3.3 ANÁLISE COMPARATIVA

A validação da eficácia do algoritmo proposto será realizada por meio de uma análise comparativa rigorosa, executada em cenários de alta carga. Para estabelecer uma condição de teste relevante, o sistema será submetido a uma carga de trabalho crescente, gerada pelo script cliente que realiza múltiplas requisições paralelas. Este processo será monitorado empiricamente através do player de vídeo final, e o ponto de sobrecarga será definido como o momento em que a degradação da experiência do usuário (QoE), manifestada por travamentos visíveis, torna-se notável. Este mesmo nível de sobrecarga será então utilizado como linha de base para todos os testes comparativos, garantindo a consistência das avaliações.

A metodologia de avaliação será centrada em métricas quantitativas que impactam diretamente a QoE do usuário final. Durante a execução de cada cenário de teste, serão coletados dados sobre: o número total de travamentos (*stalls*) do vídeo, a duração média de cada travamento e o tempo total de ociosidade do serviço (soma de todas as pausas). Essas métricas oferecem

uma visão objetiva do desempenho de cada estratégia de balanceamento sob estresse.

O algoritmo proposto, baseado na interpretação de dados obsoletos, será comparado com duas abordagens clássicas e amplamente utilizadas: *Round-Robin* e Seleção Aleatória. Para assegurar uma comparação justa, a mesma aplicação de balanceamento de carga desenvolvida em Rust será utilizada em todos os testes, alterando-se apenas o módulo de decisão do algoritmo. Serão executados os seguintes cenários:

1. Balanceamento com algoritmo Round-Robin.
2. Balanceamento com algoritmo de Seleção Aleatória.
3. Balanceamento com o algoritmo proposto, utilizando um intervalo médio de atualização dos dados dos servidores de 6 segundos.
4. Balanceamento com o algoritmo proposto, utilizando um intervalo médio de atualização dos dados dos servidores de 30 segundos.

A hipótese central é que o algoritmo proposto, mesmo com informações moderadamente desatualizadas (no intervalo de 30s), apresentará um desempenho superior em termos de QoE quando comparado aos métodos Round-Robin e Aleatório. Adicionalmente, espera-se que, entre as variações do algoritmo proposto, o intervalo de 6 segundos forneça os melhores resultados, enquanto o intervalo de 30 segundos demonstrará a capacidade do algoritmo de manter bom desempenho mesmo com dados menos atualizados.

4 DESCRIÇÃO DO EXPERIMENTO

Este capítulo apresenta uma descrição detalhada do ambiente experimental utilizado para validação da proposta, incluindo a configuração do ambiente de testes, a definição dos módulos principais e auxiliares do sistema, os detalhes de implementação de cada componente, o processo de geração de conteúdo multimídia e, por fim, a especificação dos cenários de teste utilizados para a análise comparativa dos algoritmos de balanceamento de carga.

4.1 AMBIENTE DE EXPERIMENTAÇÃO

O ambiente experimental foi configurado para proporcionar um cenário controlado e reproduzível, capaz de simular condições realistas de operação de um sistema de distribuição de conteúdo. A infraestrutura foi estabelecida em uma única máquina física operando o sistema operacional Windows, com o subsistema Windows para Linux (WSL2) configurado para execução de um *kernel* Linux completo. Esta escolha arquitetural permite a utilização de recursos nativos do Linux, como o sistema de grupos de controle (*cgroups*), essenciais para o monitoramento e limitação de recursos computacionais dos contêineres.

A plataforma de containerização Docker foi empregada como ambiente de execução para todos os componentes do sistema. O Docker oferece isolamento de processos, gerenciamento de recursos e padronização do ambiente de execução, garantindo que os experimentos possam ser reproduzidos de forma consistente. Além disso, a utilização de contêineres elimina problemas relacionados a dependências de bibliotecas e configurações de sistema operacional, assegurando que cada componente opere em um ambiente idêntico e controlado.

4.1.1 Particularidades do Docker e *cgroups*

O Docker utiliza o sistema de *cgroups* (grupos de controle) do kernel Linux para implementar limites de recursos e realizar o monitoramento de uso de CPU, memória e entrada/saída de disco. Especificamente, o experimento utiliza *cgroups v2*, que oferece uma interface unificada e simplificada para o gerenciamento de recursos. Através dos arquivos de sistema disponibilizados pelo *cgroups*, é possível ler diretamente as métricas de consumo de memória e taxa de leitura de disco de cada contêiner em tempo real.

Para garantir a consistência entre as execuções dos experimentos e eliminar o efeito de *cache* do sistema operacional nos resultados, é fundamental realizar a limpeza do *cache* de páginas do kernel Linux antes de cada execução. Esta operação é realizada através do comando:

```
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Este comando instrui o kernel a liberar os *caches* de página (*pagecache*), *dentries* e *inodes*, garantindo que cada execução do experimento parta de um estado limpo, sem dados

previamente armazenados em memória. A operação requer privilégios de superusuário (`sudo`) e é executada antes de cada bateria de testes.

O Docker também permite a configuração de limites explícitos de recursos para cada contêiner através de parâmetros na configuração do *Docker Compose*. No experimento, foram configurados limites de memória RAM (através do parâmetro `memory`) e limites de taxa de leitura de disco (através do parâmetro `device_read_bps`). Estes limites são fundamentais para simular servidores com capacidades heterogêneas, permitindo avaliar o comportamento do algoritmo de balanceamento em cenários com recursos desiguais.

4.1.2 Organização do conteúdo e bind mounts

Para simular o cenário realista de servidores de distribuição de conteúdo com arquivos exclusivos, foi necessário replicar o conteúdo de vídeo em múltiplos diretórios no sistema de arquivos do host WSL. O conteúdo multimídia foi armazenado em um diretório global localizado em `/data` no sistema de arquivos Linux do WSL, com subdiretórios específicos para cada servidor (`/data/server-1`, `/data/server-2`, e assim por diante até `/data/server-8`).

Cada contêiner de servidor MPEG-DASH foi configurado com um *bind mount* que mapeia o diretório específico do servidor no host para o diretório interno do contêiner. Esta técnica permite que cada servidor acesse apenas seu próprio conjunto de arquivos, simulando a exclusividade de conteúdo encontrada em servidores de borda de uma CDN real. O uso de *bind mounts* também evita a necessidade de copiar os arquivos para dentro das imagens Docker, reduzindo o tamanho das imagens e facilitando a atualização do conteúdo sem reconstruir os contêineres.

4.2 DEFINIÇÃO DOS MÓDULOS

O sistema experimental é composto por módulos principais e auxiliares, cada um desempenhando uma função específica na arquitetura do experimento. Esta seção descreve a organização e o propósito de cada componente.

4.2.1 Módulos Principais

Os módulos principais formam o núcleo funcional do sistema de distribuição de conteúdo e são responsáveis pelo processamento e encaminhamento das requisições dos clientes:

1. **Servidores MPEG-DASH:** Conjunto de 8 instâncias de servidores web responsáveis por servir o conteúdo de vídeo segmentado de acordo com o padrão MPEG-DASH. Cada servidor é executado em um contêiner Docker isolado e possui limites específicos de memória e taxa de leitura de disco, permitindo a simulação de servidores com capacidades heterogêneas. Os servidores também são responsáveis por coletar e publicar suas próprias métricas de consumo de recursos.

2. **Balanceador de Carga:** Componente central do experimento, responsável por receber todas as requisições HTTP dos clientes e distribuí-las entre os servidores MPEG-DASH disponíveis de acordo com a estratégia de balanceamento configurada. O balanceador opera como um *proxy* reverso HTTP e é implementado para suportar múltiplas estratégias de seleção de servidor, permitindo a comparação entre diferentes algoritmos.
3. **Gerador de Carga:** Aplicação cliente que simula múltiplos usuários acessando o sistema de forma concorrente. O gerador é parametrizável em termos de número de usuários simultâneos e duração do teste, permitindo a criação de cenários de carga variados para avaliar o comportamento do sistema sob diferentes níveis de estresse.

4.2.2 Módulos Auxiliares

Os módulos auxiliares fornecem serviços de suporte necessários para o funcionamento e monitoramento do sistema:

1. **Broker MQTT (NanoMQ):** Serviço de mensageria que implementa o protocolo MQTT (*Message Queuing Telemetry Transport*), um protocolo leve de comunicação *publish-subscribe*. O *broker* atua como intermediário, recebendo as métricas publicadas pelos servidores MPEG-DASH e disponibilizando-as para o balanceador de carga através de assinaturas em tópicos específicos. O uso do MQTT permite a comunicação assíncrona e desacoplada entre os componentes.
2. **Painel de Monitoramento em Tempo Real:** Interface web que oferece visualização gráfica e em tempo real das métricas de cada servidor e do balanceador de carga. O painel permite acompanhar o comportamento do sistema durante os experimentos, facilitando a detecção de anomalias e a compreensão do comportamento dinâmico do algoritmo de balanceamento.
3. **Coletor de Métricas (API):** Serviço Python que expõe uma API REST para consulta das métricas dos contêineres através do acesso direto aos arquivos do *cgroups*. Este componente complementa o monitoramento realizado internamente pelos servidores, permitindo a validação cruzada das métricas e fornecendo dados para o painel de monitoramento.

4.3 DETALHES DE IMPLEMENTAÇÃO DOS MÓDULOS

Esta seção apresenta os detalhes técnicos de implementação de cada módulo do sistema, incluindo as tecnologias empregadas, as principais funcionalidades e os trechos de código mais relevantes.

4.3.1 Servidores MPEG-DASH

Os servidores MPEG-DASH foram implementados utilizando ASP.NET Core 8.0, uma plataforma de desenvolvimento web moderna e de alto desempenho da Microsoft. A escolha desta tecnologia foi motivada por sua capacidade nativa de servir arquivos estáticos de forma eficiente, sua arquitetura assíncrona que permite alto nível de concorrência e a disponibilidade de bibliotecas para comunicação MQTT.

Cada servidor é configurado como uma aplicação web mínima que serve arquivos estáticos do diretório `wwwroot/Static`, onde estão armazenados os segmentos de vídeo MPEG-DASH. A aplicação não realiza processamento de vídeo em tempo real; ela apenas entrega os arquivos pré-processados aos clientes que os solicitam.

4.3.1.1 Monitoramento e Publicação de Métricas

O componente mais crítico da implementação dos servidores é o sistema de monitoramento de recursos e publicação de métricas. Esta funcionalidade é implementada através da classe `MetricsPublisher`, um serviço de segundo plano (*background service*) que executa continuamente enquanto o servidor está ativo.

O `MetricsPublisher` realiza periodicamente as seguintes operações:

1. Leitura do consumo atual de memória através do arquivo `/sys/fs/cgroup/memory.current` do `cgroups v2`;
2. Leitura das estatísticas de entrada/saída através do arquivo `/sys/fs/cgroup/io.stat`, extraindo o total de *bytes* lidos (`rbytes`);
3. Cálculo da taxa de leitura de disco através da diferença entre leituras consecutivas dividida pelo intervalo de tempo decorrido;
4. Normalização das métricas em relação aos limites configurados para o contêiner, gerando valores entre 0 e 1;
5. Publicação das métricas normalizadas em um tópico MQTT para consumo pelo balanceador de carga.

O intervalo de publicação é configurável através da variável de ambiente `METRICS_PUBLISH_INTERVAL`, permitindo avaliar o impacto da frequência de atualização das métricas no desempenho do algoritmo de balanceamento.

A estrutura de dados publicada contém os seguintes campos:

- `memory_current`: Consumo normalizado de memória (0 a 1);
- `disk_read`: Taxa normalizada de leitura de disco (0 a 1);

- `active_request_count`: Número de requisições HTTP ativas sendo processadas no momento;
- `timestamp_unix`: *Timestamp* Unix da coleta.

O contador de requisições ativas é implementado através da classe `RequestCounter`, que utiliza operações atômicas para incrementar e decrementar o contador de forma *thread-safe*, garantindo a consistência dos dados mesmo sob alta concorrência.

4.3.2 Balanceador de Carga

O balanceador de carga foi implementado em Rust, uma linguagem de programação de sistemas que oferece garantias de segurança de memória em tempo de compilação e desempenho comparável a linguagens como C e C++. A escolha do Rust foi motivada pela necessidade de alto desempenho e baixa latência no processamento das requisições, características essenciais para um componente crítico como o balanceador de carga.

4.3.2.1 Arquitetura e Funcionamento

O balanceador opera como um *proxy* reverso HTTP, recebendo requisições dos clientes na porta 8080 e encaminhando-as para um dos servidores MPEG-DASH disponíveis. Um aspecto fundamental da implementação é que o balanceamento ocorre ao nível de cada requisição HTTP individual. Isso significa que, durante uma única sessão de reprodução MPEG-DASH, diferentes requisições (como a solicitação do manifesto MPD, dos segmentos de inicialização e dos segmentos de mídia) podem ser atendidas por servidores diferentes, dependendo das decisões tomadas pelo algoritmo de balanceamento a cada momento.

A arquitetura do balanceador é baseada no padrão de projeto *Strategy*, que permite a fácil substituição do algoritmo de seleção de servidor sem modificar o código principal do balanceador. O *trait* `ServerSelectionStrategy` define a interface que deve ser implementada por qualquer estratégia de balanceamento:

```
pub trait ServerSelectionStrategy: Send + Sync {
    fn pick_server(&self, servers: &[String]) -> Option<String>;
}
```

Esta abstração permite que o experimento compare diferentes algoritmos de balanceamento utilizando exatamente a mesma infraestrutura, garantindo que as diferenças observadas nos resultados sejam devidas exclusivamente ao algoritmo e não a variações na implementação.

4.3.2.2 Estratégias de Balanceamento Implementadas

Três estratégias de balanceamento foram implementadas para fins de comparação:

1. **Round-Robin:** Estratégia determinística que seleciona os servidores de forma cíclica, garantindo uma distribuição uniforme das requisições ao longo do tempo. A implementação utiliza um contador atômico que é incrementado a cada seleção.
2. **Seleção Aleatória:** Estratégia não-determinística que seleciona um servidor de forma uniformemente aleatória entre os disponíveis. Esta estratégia serve como linha de base para avaliar o ganho proporcionado por estratégias mais sofisticadas.
3. **Monitoramento de Memória:** Estratégia proposta neste trabalho, que utiliza as métricas de consumo de memória e disco publicadas pelos servidores para calcular probabilidades de seleção baseadas na carga atual e na interpretação de dados potencialmente obsoletos, conforme descrito no Capítulo 3.

A estratégia de monitoramento de memória mantém uma estrutura de dados interna com as métricas mais recentes de cada servidor, que é atualizada através de uma assinatura MQTT no tópico `loadbalancer/metrics`. Quando uma nova requisição chega, o algoritmo calcula as probabilidades de seleção de acordo com as Equações 1 e 2, e realiza uma seleção aleatória ponderada baseada nessas probabilidades.

4.3.3 Gerador de Carga

O gerador de carga é uma aplicação Python que utiliza a biblioteca `aiohttp` para realizar requisições HTTP assíncronas de alto desempenho. A implementação é baseada em um modelo de concorrência com múltiplas *coroutines* assíncronas (*workers*), onde cada *worker* simula o comportamento de um cliente MPEG-DASH real.

4.3.3.1 Simulação de Clientes MPEG-DASH

Cada *worker* do gerador de carga executa o seguinte fluxo de operações, que emula o comportamento de um *player* de vídeo MPEG-DASH:

1. **Seleção de Conteúdo:** O *worker* seleciona aleatoriamente um dos diretórios de vídeo disponíveis (de `video-1` até `video-4`, de acordo com a configuração).
2. **Requisição do Manifesto MPD:** A primeira requisição HTTP é feita para obter o arquivo MPD (*Media Presentation Description*), que é um documento XML contendo metadados sobre o vídeo, incluindo as resoluções disponíveis, os *bitrates*, a duração dos segmentos e os *templates* de URL para acessar cada segmento.
3. **Parsing do Manifesto:** O arquivo MPD é analisado utilizando a biblioteca padrão `xml.etree.Element` extraindo as informações sobre os segmentos de inicialização e os segmentos de mídia de todas as representações (resoluções) disponíveis.

4. **Requisição dos Segmentos de Inicialização:** Para cada representação (resolução) do vídeo, o *worker* solicita o segmento de inicialização correspondente, que contém informações de *codec* e configuração necessárias para decodificar os segmentos subsequentes.
5. **Requisição Sequencial dos Segmentos de Mídia:** O *worker* então solicita sequencialmente os segmentos de mídia, aguardando a conclusão de cada requisição HTTP antes de iniciar a próxima. Este comportamento, sem *buffering* antecipado, simula um cenário onde o cliente consome o conteúdo em tempo real, sem realizar *download* prévio de múltiplos segmentos. Esta é uma simulação simplificada que não representa exatamente o comportamento de *players* MPEG-DASH modernos, que geralmente implementam estratégias de *buffering* e seleção adaptativa de qualidade, mas é adequada para gerar carga no sistema e avaliar o desempenho do balanceador.

O número de *workers* concorrentes é parametrizável através do argumento de linha de comando `-concurrent`, permitindo simular desde cenários de baixa carga (dezenas de usuários) até cenários de alta carga (milhares de usuários simultâneos). A duração do teste também é configurável através do parâmetro `-duration`, permitindo controlar o tempo total de execução do experimento.

4.3.3.2 Aplicação Cliente para Validação e Visualização

Além do gerador de carga automatizado, foi desenvolvida uma aplicação cliente baseada em HTML (`test-dash.html`) que utiliza a biblioteca `dash.js` para reprodução real de vídeos MPEG-DASH com visualização em tempo real das métricas de QoE. Esta aplicação serve como ferramenta de validação e depuração, permitindo observar o comportamento do sistema durante a reprodução de vídeo e coletar dados detalhados sobre a qualidade da experiência do usuário.

A aplicação implementa um *player* de vídeo completo com as seguintes funcionalidades:

1. **Gerenciamento de Sessão de Vídeo:** A aplicação inicializa o *player* `dash.js` com configurações específicas de *buffer* (`stableBufferTime`, `initialBufferLevel`) e registra *handlers* para eventos de reprodução (`PLAYBACK_STARTED`, `PLAYBACK_WAITING`, `PLAYBACK_PLAYING`, `PLAYBACK_ENDED`). Esses eventos permitem rastrear o ciclo de vida completo da sessão de reprodução, incluindo o início do *streaming*, momentos de travamento (*stalls*), retomada da reprodução e finalização do vídeo.
2. **Visualização em Tempo Real:** Três gráficos interativos são atualizados dinamicamente durante a reprodução do vídeo, utilizando a biblioteca `Chart.js`:
 - **Gráfico de Qualidade (*Quality Timeline*):** Exibe a evolução da qualidade (resolução e *bitrate*) selecionada pelo algoritmo de adaptação do `dash.js` ao longo dos segmentos de vídeo reproduzidos. O eixo Y representa as diferentes resoluções

disponíveis (144P a 1080P), e o eixo X representa a sequência de segmentos, permitindo visualizar as mudanças de qualidade durante a sessão.

- **Gráfico de *Bitrate* (*Bitrate Timeline*):** Mostra a taxa de transferência média (*throughput*) medida pelo `dash.js` para cada segmento baixado, em kilobits por segundo (kbps). Este gráfico utiliza uma escala logarítmica no eixo Y para melhor visualização da variação de *bitrate* ao longo do tempo.
- **Gráfico de Travamentos (*Stall Timeline*):** Apresenta um gráfico de dispersão onde cada ponto representa um evento de travamento (*stall*), com a posição no eixo X indicando o segmento onde ocorreu o travamento e o eixo Y mostrando a duração do travamento em segundos. Este gráfico permite identificar visualmente a frequência e a severidade dos travamentos durante a sessão.

3. **Coleta de Dados para Análise:** Ao final de cada sessão de reprodução, a aplicação exporta automaticamente para o console do navegador arrays estruturados contendo todos os dados coletados:

- Sequência de qualidades reproduzidas (IDs de representação e *labels* descritivos);
- Sequência de *bitrates* medidos para cada segmento;
- Lista detalhada de travamentos com timestamp, duração e índice do segmento;
- Estatísticas agregadas: número total de travamentos e tempo total de travamento acumulado.

Estes dados exportados podem ser facilmente copiados do console e processados posteriormente para análise estatística e geração de gráficos comparativos entre diferentes cenários de teste.

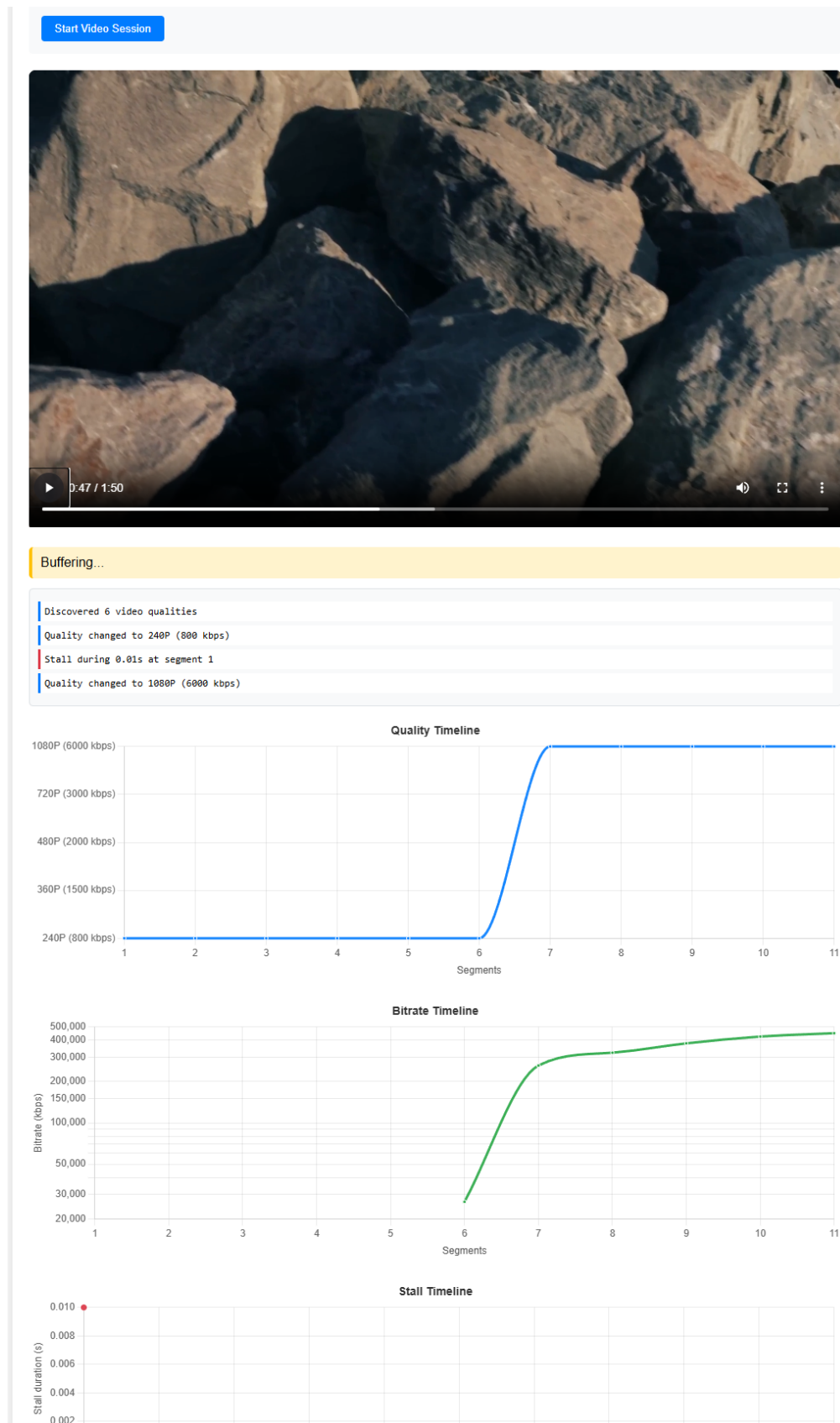
A Figura 3 apresenta a interface da aplicação cliente durante a reprodução de um vídeo, mostrando o *player* de vídeo e os três gráficos de monitoramento em tempo real, que permitem a validação visual do comportamento do sistema e a identificação imediata de problemas de QoE.

4.3.4 Broker MQTT (NanoMQ)

O *broker* MQTT utilizado no experimento é o NanoMQ, uma implementação leve e de alto desempenho do protocolo MQTT desenvolvida pela EMQ Technologies. O NanoMQ foi escolhido por sua simplicidade de configuração, baixo consumo de recursos e conformidade completa com o padrão MQTT 5.0.

O *broker* é executado em um contêiner Docker dedicado e expõe a porta 1883 para comunicação MQTT não criptografada. A configuração é realizada através de um arquivo `nanomq.conf` montado como volume no contêiner, permitindo ajustes finos de parâmetros como o número máximo de conexões simultâneas e o tamanho dos *buffers* de mensagens.

Figura 3 – Aplicação cliente MPEG-DASH com visualização em tempo real de métricas de QoE: gráfico de qualidade, gráfico de *bitrate* e gráfico de travamentos.



Fonte: Elaborado pelo autor (2025)

No contexto do experimento, o *broker* atua como canal de comunicação entre os servidores MPEG-DASH (publicadores) e o balanceador de carga (assinante), permitindo que as métricas sejam transmitidas de forma assíncrona e desacoplada, sem que os servidores precisem conhecer o endereço do balanceador ou vice-versa.

4.3.5 Orquestração com Docker Compose

A orquestração de todos os componentes do sistema é realizada através do Docker Compose, uma ferramenta para definir e executar aplicações Docker multi-contêiner. O arquivo `docker-compose.yml` descreve de forma declarativa todos os serviços, suas configurações, limites de recursos, volumes e dependências.

Cada servidor MPEG-DASH é definido como um serviço separado com suas próprias configurações de recursos. Por exemplo, o servidor 1 é configurado com 4096 MB de memória e taxa máxima de leitura de disco de 512 MB/s, enquanto o servidor 8 possui apenas 512 MB de memória e 4 MB/s de leitura de disco. Esta heterogeneidade é fundamental para avaliar a capacidade do algoritmo de balanceamento em distribuir a carga de forma proporcional à capacidade de cada servidor.

Os comandos principais para gerenciar o ambiente experimental são:

```
# Limpar cache e iniciar todos os serviços
echo 3 | sudo tee /proc/sys/vm/drop_caches && \
docker compose down && \
docker compose up -d --build

# Visualizar logs do balanceador
docker compose logs -f load-balancer

# Encerrar todos os serviços
docker compose down
```

4.3.6 Painel de Monitoramento

O painel de monitoramento foi implementado como uma página HTML estática que utiliza a biblioteca Chart.js para renderizar gráficos em tempo real. A página é servida através de um contêiner nginx e se comunica com a API de métricas através de requisições HTTP periódicas utilizando JavaScript.

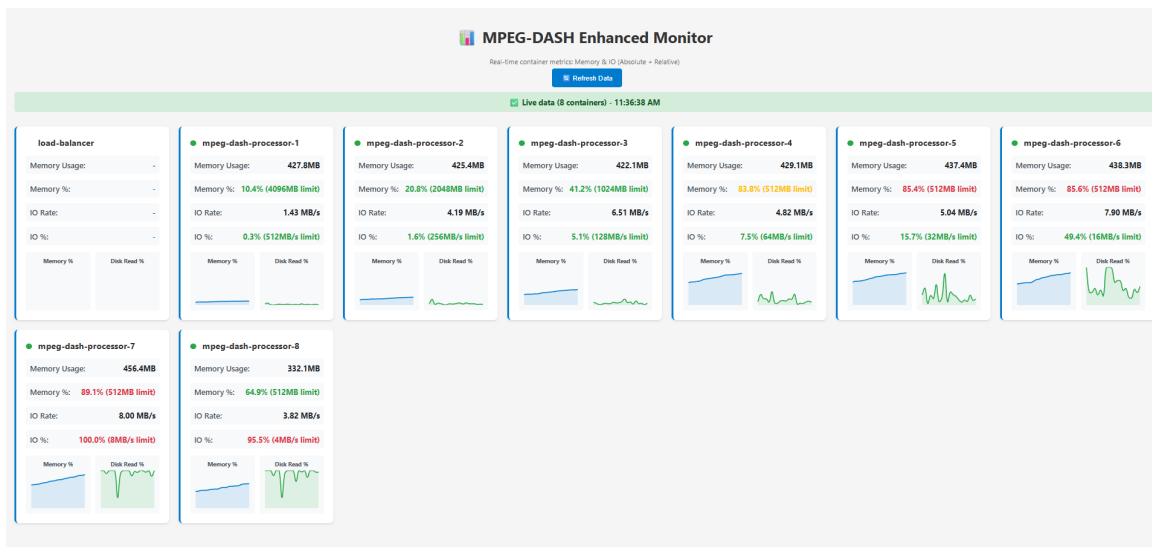
O painel exibe, para cada servidor e para o balanceador de carga:

- Consumo de memória (absoluto e percentual);
- Taxa de leitura de disco (MB/s e percentual);

- Número de requisições ativas;
- Gráficos de séries temporais mostrando a evolução das métricas.

O painel é acessível através de um navegador web na porta 3001 do host e foi utilizado durante o desenvolvimento e depuração do sistema para validar o comportamento dos componentes e identificar problemas de configuração. A Figura 4 apresenta a interface do painel de monitoramento, mostrando as métricas de múltiplos servidores MPEG-DASH em tempo real.

Figura 4 – Painel de monitoramento em tempo real mostrando métricas de consumo de memória, taxa de leitura de disco e requisições ativas dos servidores MPEG-DASH.



Fonte: Elaborado pelo autor (2025)

4.4 GERAÇÃO DE CONTEÚDO

O conteúdo multimídia utilizado nos experimentos foi gerado utilizando o `ffmpeg`, uma ferramenta de código aberto amplamente utilizada para processamento de áudio e vídeo. O `ffmpeg` é capaz de codificar, decodificar, transcodificar, multiplexar e aplicar filtros em arquivos multimídia, sendo a ferramenta padrão da indústria para preparação de conteúdo para *streaming*.

4.4.1 Codificação e Segmentação MPEG-DASH

O processo de geração do conteúdo MPEG-DASH envolveu a conversão de um arquivo de vídeo de entrada para o formato MPEG-DASH com múltiplas representações (resoluções e *bitrates*). O comando utilizado foi:

```
ffmpeg -y -i video.mp4 \
-filter_complex "[0:v]format=yuv420p,setsar=1,split=6[v0][v1][v2][v3][v4][v5]; \
[v0]scale=1920:1080:flags=lanczos[v1080]; \
[v1]scale=1280:720:flags=lanczos[v720]; \
```

```

        [v2]scale=854:480:flags=lanczos[v480]; \
        [v3]scale=640:360:flags=lanczos[v360]; \
        [v4]scale=426:240:flags=lanczos[v240]; \
        [v5]scale=256:144:flags=lanczos[v144]" \
-map "[v1080]" -map "[v720]" -map "[v480]" -map "[v360]" \
    -map "[v240]" -map "[v144]" -map 0:a:0 \
-c:v:0 libx264 -b:v:0 6000k -maxrate:v:0 6600k -bufsize:v:0 12000k \
-c:v:1 libx264 -b:v:1 3000k -maxrate:v:1 3300k -bufsize:v:1 6000k \
-c:v:2 libx264 -b:v:2 2000k -maxrate:v:2 2200k -bufsize:v:2 4000k \
-c:v:3 libx264 -b:v:3 1500k -maxrate:v:3 1650k -bufsize:v:3 3000k \
-c:v:4 libx264 -b:v:4 800k -maxrate:v:4 880k -bufsize:v:4 1600k \
-c:v:5 libx264 -b:v:5 300k -maxrate:v:5 330k -bufsize:v:5 600k \
-profile:v:0 high -profile:v:1 main -profile:v:2 main \
-profile:v:3 main -profile:v:4 main -profile:v:5 main \
-g 120 -keyint_min 120 -sc_threshold 0 -pix_fmt yuv420p \
-preset veryfast \
-c:a aac -b:a 128k -ar 48000 -ac 2 \
-use_template 1 -use_timeline 1 -seg_duration 4 \
-init_seg_name 'init-stream$RepresentationID$.m4s' \
-media_seg_name 'chunk-stream$RepresentationID$-$Number%05d$.m4s' \
-adaptation_sets "id=0,streams=v id=1,streams=a" \
manifest.mpd

```

Este comando realiza as seguintes operações:

1. **Processamento de Vídeo:** Utiliza um *filter complex* para criar 6 fluxos de vídeo distintos a partir do vídeo de entrada. O filtro `split` duplica o fluxo de entrada, e cada cópia é redimensionada para uma resolução específica utilizando o algoritmo de escala Lanczos, que oferece alta qualidade visual.
2. **Resoluções e Bitrates:** São geradas 6 representações de vídeo:
 - Full HD (1920×1080) a 6000 kbps
 - HD (1280×720) a 3000 kbps
 - 854×480 a 2000 kbps
 - 640×360 a 1500 kbps
 - 426×240 a 800 kbps
 - 256×144 a 300 kbps

3. **Codificação:** Cada fluxo de vídeo é codificado utilizando o *codec* H.264 (através da biblioteca `libx264`) com parâmetros específicos de *bitrate* alvo, *bitrate* máximo e tamanho de *buffer*. O perfil H.264 high é utilizado para a resolução Full HD, enquanto o perfil main é usado para as demais resoluções.
4. **Segmentação:** O vídeo é dividido em segmentos de 4 segundos cada (`seg_duration 4`), um valor típico para aplicações de *streaming* adaptativo que oferece um bom equilíbrio entre a granularidade da adaptação e a eficiência de transferência.
5. **Áudio:** O áudio é codificado em AAC a 128 kbps com taxa de amostragem de 48 kHz e 2 canais (estéreo).
6. **Manifesto MPD:** É gerado um arquivo `manifest.mpd` que descreve a estrutura do conteúdo, incluindo as resoluções disponíveis, os *bitrates*, a duração dos segmentos e os *templates* de URL para acessar os arquivos de inicialização e os segmentos de mídia.

4.4.2 Replicação e Distribuição do Conteúdo

Após a geração do conteúdo MPEG-DASH, os arquivos resultantes (manifesto MPD, segmentos de inicialização e segmentos de mídia) foram copiados para um diretório global no sistema de arquivos WSL localizado em `/data`. Para simular servidores com conteúdo exclusivo, o conteúdo foi replicado em múltiplos subdiretórios utilizando o comando `rsync`:

```
sudo rsync -a --progress ./server-1/ ./server-2/
sudo rsync -a --progress ./server-1/ ./server-3/
# ... continuando até server-8
```

Cada subdiretório `/data/server-X` foi então montado como um volume no contêiner correspondente através da diretiva `volumes` no arquivo `docker-compose.yml`:

```
volumes:
  - /data/server-1:/app/wwwroot/Static
```

Este mecanismo de *bind mount* permite que cada servidor acesse apenas seu próprio conjunto de arquivos, simulando o cenário realista de uma CDN onde cada servidor de borda mantém uma cópia local do conteúdo. A utilização de *bind mounts* também facilita a atualização do conteúdo sem necessidade de reconstruir as imagens Docker, uma vez que as modificações no diretório do host são imediatamente refletidas dentro dos contêineres.

4.5 CENÁRIOS DE TESTE

A avaliação experimental do algoritmo proposto foi conduzida através de múltiplos cenários de teste, projetados para avaliar o desempenho do sistema sob diferentes condições de

carga e configurações de recursos dos servidores. Os cenários foram cuidadosamente definidos para permitir uma análise comparativa rigorosa entre as diferentes estratégias de balanceamento de carga.

4.5.1 Parâmetros Globais

Todos os cenários de teste compartilham os seguintes parâmetros:

- **Timeout de requisição:** 30 segundos
- **Número de vídeos no catálogo:** 12
- **Número de resoluções disponíveis:** 6 (conforme descrito na seção anterior)

4.5.2 Definição dos Cenários

Foram definidos quatro cenários principais, cada um testado com três estratégias de balanceamento diferentes (Round-Robin, Seleção Aleatória e Monitoramento de Memória com diferentes intervalos de atualização). A Tabela 1 apresenta a especificação detalhada de cada cenário.

Tabela 1 – Cenários de teste definidos para avaliação experimental

Cenário	Usuários	Distribuição	Configuração dos Servidores
1	100	Heterogênea	S1: 4096MB / 512MB/s; S2: 2048MB / 256MB/s; S3: 1024MB / 128MB/s; S4: 512MB / 64MB/s; S5: 512MB / 32MB/s; S6: 512MB / 16MB/s; S7: 512MB / 8MB/s; S8: 512MB / 4MB/s
2	100	Homogênea	Todos os servidores: 1024MB RAM e 256MB/s I/O
3	1000	Heterogênea	Mesma configuração do Cenário 1
4	1000	Homogênea	Mesma configuração do Cenário 2

4.5.3 Descrição dos Cenários

4.5.3.1 Cenário 1: Baixa Concorrência com Servidores Heterogêneos

Este cenário simula uma situação de carga moderada com 100 usuários simultâneos acessando um conjunto de servidores com capacidades significativamente diferentes. A heterogeneidade dos recursos visa avaliar a capacidade do algoritmo proposto de identificar e priorizar servidores com maior disponibilidade de recursos. Espera-se que estratégias que ignoram o estado dos servidores (como Round-Robin e Seleção Aleatória) distribuam a carga de forma

uniforme, potencialmente sobrecarregando os servidores menos capazes, enquanto o algoritmo de monitoramento de memória deve ser capaz de direcionar mais requisições para os servidores com maior capacidade.

4.5.3.2 *Cenário 2: Baixa Concorrência com Servidores Homogêneos*

Este cenário mantém o mesmo nível de carga do Cenário 1, mas utiliza servidores com recursos idênticos. O objetivo é avaliar o comportamento das estratégias em um ambiente onde não há vantagem teórica em favorecer um servidor específico. Neste cenário, espera-se que todas as estratégias apresentem desempenho similar, uma vez que a distribuição uniforme de carga é adequada quando os servidores possuem capacidades equivalentes.

4.5.3.3 *Cenário 3: Alta Concorrência com Servidores Heterogêneos*

Este cenário eleva significativamente a carga do sistema para 1000 usuários simultâneos, mantendo a configuração heterogênea de servidores. O objetivo é avaliar o comportamento das estratégias sob condições de alta pressão, onde a capacidade de identificar e evitar servidores sobrecarregados torna-se crítica para manter a qualidade de serviço. Este é o cenário mais exigente e onde se espera observar as maiores diferenças de desempenho entre as estratégias.

4.5.3.4 *Cenário 4: Alta Concorrência com Servidores Homogêneos*

O último cenário combina alta carga com servidores homogêneos, oferecendo um ponto de comparação para o Cenário 3. Ele permite avaliar se as diferenças observadas no Cenário 3 são devidas à heterogeneidade dos recursos ou ao nível de carga do sistema.

4.5.4 **Estratégias de Balanceamento Avaliadas**

Para cada um dos quatro cenários descritos, foram executados testes com as seguintes estratégias de balanceamento:

1. **Round-Robin:** Estratégia de linha de base que distribui as requisições de forma cíclica entre os servidores.
2. **Seleção Aleatória:** Estratégia de linha de base que seleciona um servidor aleatoriamente para cada requisição.
3. **Monitoramento de Memória (intervalo médio de 6 segundos):** Algoritmo proposto com intervalo médio de atualização de métricas de 6 segundos, representando um cenário com informações relativamente atualizadas.
4. **Monitoramento de Memória (intervalo médio de 30 segundos):** Algoritmo proposto com intervalo médio de atualização de 30 segundos, representando um cenário com informações moderadamente desatualizadas.

Cada combinação de cenário e estratégia de balanceamento foi implementada como uma *branch* Git específica no repositório do projeto, permitindo o isolamento e a reprodutibilidade de cada configuração experimental. Esta organização através de *branches* facilitou a execução sistemática dos testes e garantiu que as diferentes configurações não interferissem entre si.

A avaliação com diferentes intervalos de atualização visa demonstrar a robustez do algoritmo proposto mesmo quando operando com informações de carga moderadamente obsoletas, uma situação comum em sistemas distribuídos de larga escala onde o custo de monitoramento muito frequente pode ser proibitivo.

4.5.5 Métricas de Avaliação

Para cada combinação de cenário e estratégia de balanceamento, foram coletadas as seguintes métricas de Qualidade de Experiência (QoE):

- Número total de travamentos (*stalls*) observados durante a reprodução do vídeo;
- Duração média de cada travamento em milissegundos;
- Tempo total de ociosidade (soma de todas as pausas) em segundos;
- Taxa de erros de requisições HTTP (percentual de requisições que falharam);
- Latência média de resposta das requisições em milissegundos.

Estas métricas foram escolhidas por seu impacto direto na experiência percebida pelo usuário final ao consumir conteúdo de vídeo em *streaming*. A análise comparativa destas métricas entre as diferentes estratégias permite quantificar objetivamente o benefício (ou eventual desvantagem) do algoritmo proposto em relação às abordagens convencionais.

5 CONSIDERAÇÕES PARCIAIS

O presente trabalho propõe investigar e desenvolver uma solução para balanceamento de carga em Sistemas de Distribuição de Conteúdo, um componente crítico para garantir Qualidade de Experiência (QoE) ao usuário final. O trabalho identificou que estratégias convencionais de balanceamento, como *Round-Robin* ou Seleção Aleatória, frequentemente ignoram o estado real dos recursos dos servidores, especialmente a memória, que é um fator de gargalo em sistemas cuja operação principal é a recuperação e entrega de dados oriundos de arquivos persistidos em memória secundária.

Diante dessa lacuna, foi elaborado um algoritmo de balanceamento probabilístico que integra o monitoramento sistemático de memória principal e secundária como fator decisório. A proposta adapta o conceito de *Load Interpretation* para ajustar a distribuição de carga com base não apenas nas métricas de consumo de recursos (isto é, nesse caso, memória principal e secundária), mas também na idade dessas informações, evitando assim os efeitos adversos de dados desatualizados. A carga de cada servidor (L_i) é calculada como uma média ponderada do consumo normalizado de memória RAM (LM_i) e da carga de disco (LD_i), conforme a Equação 2. A probabilidade de um servidor ser escolhido (P_i) é então determinada dinamicamente, de modo a equalizar a carga projetada entre todos os nós do sistema.

A arquitetura do sistema para a fase experimental foi definida para operar em um ambiente controlado e reproduzível com contêineres Docker, utilizando servidores MPEG-DASH, um *broker* MQTT para a comunicação assíncrona dos dados de monitoramento e um balanceador de carga em Rust. A validação da proposta será realizada por meio de uma análise comparativa rigorosa com os algoritmos *Round-Robin* e Seleção Aleatória, sob cenários de alta carga, utilizando métricas de QoE como o número e a duração de travamentos do vídeo.

Com a fundamentação teórica e a arquitetura propostas, o trabalho está preparado para a fase de implementação e validação empírica. As atividades subsequentes estão organizadas para construir, testar e avaliar sistematicamente cada componente, culminando na análise final que determinará a eficácia do algoritmo proposto em otimizar a distribuição de conteúdo de maneira inteligente e resiliente.

5.1 PRÓXIMOS PASSOS

Esta seção descreve as atividades que serão feitas a seguir, bem como o planejamento de execução.

1. Configuração e testes do sistema gerenciador de filas (*broker* MQTT);
2. Implementação e configuração inicial do servidor MPEG-DASH
 - 2.1. Configuração e testes com o *player* enviando diretamente requisições para o servidor;

- 2.2. Instrumentação do servidor, enviando publicações periódicas ao serviço gerenciador de filas com as informações necessárias;
3. Implementação do balanceador de carga;
 - 3.1. Criação da arquitetura de código flexível para escolher o algoritmo usado no balanceamento;
 - 3.2. Implementação e teste do balanceador com servidores falsos e algoritmos simples;
 - 3.3. Implementação do algoritmo proposto no balanceador de carga;
 - 3.4. Testes do balanceador de carga ligado nos servidores reais;
4. Criação da aplicação cliente geradora de sobrecarga no sistema;
 - 4.1. Teste da aplicação com um servidor MPEG-DASH público;
 - 4.2. Teste da aplicação ligada em um dos servidores criados (diretamente - sem balanceador de carga);
 - 4.3. Teste da aplicação ligada no balanceador de carga com todos os servidores configurados;
5. Configuração do *player* simples no navegador;
6. Execução dos testes de qualidade com o *player* simples;
7. Elaboração dos gráficos e descrição dos resultados finais;

5.2 CRONOGRAMA

Atividades	Meses						
	Jun	Jul	Ago	Set	Out	Nov	Dez
1							
2							
3							
4							
5							
6							
7							

REFERÊNCIAS

ADEWOJO, Adekunbi; BASS, Julian. A novel weight-assignment load balancing algorithm for cloud applications. **SN Computer Science**, v. 4, 03 2023. Citado na página 13.

ALI, Waris; FANG, Chao; KHAN, Akmal. A survey on the state-of-the-art cdn architectures and future directions. **Journal of Network and Computer Applications**, v. 236, 2025. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804525000037>. Citado na página 11.

Apple Inc. **About the Virtual Memory System**. 2012. Documentação de arquivo que descreve os conceitos fundamentais do sistema de memória virtual do macOS. Disponível em: <https://developer.apple.com/library/archive/documentation/Performance/Conceptual/ManagingMemory/Articles/AboutMemory.html>. Citado na página 25.

Apple Inc. **IOKit Fundamentals: Introduction**. 2013. Documentação de arquivo que apresenta o framework I/O Kit, fundamental para o monitoramento de dispositivos, incluindo I/O de disco. Disponível em: <https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/Introduction/Introduction.html>. Citado na página 25.

ARPACI-DUSSEAU, R.H. et al. The architectural costs of streaming i/o: A comparison of workstations, clusters, and smps. In: **Proceedings 1998 Fourth International Symposium on High-Performance Computer Architecture**. [S.l.: s.n.], 1998. Citado na página 11.

BEGEN, Ali; AKGUL, Tankut; BAUGHER, Mark. Watching video over the web: Part 1: Streaming protocols. **IEEE Internet Computing**, v. 15, n. 2, 2011. Citado 2 vezes nas páginas 19 e 20.

BELGAUM, Mohammad Riyaz et al. A systematic review of load balancing techniques in software-defined networking. **IEEE Access**, v. 8, 2020. Citado na página 18.

BELLA, Mochamad Rexa Mei; DATA, Mahendra; YAHYA, Widhi. Web server load balancing based on memory utilization using docker swarm. In: **2018 International Conference on Sustainable Information Engineering and Technology (SIET)**. [S.l.: s.n.], 2018. Citado na página 13.

BLIAL, Othmane; MAMOUN, Mouad Ben; BENAINI, Redouane. An overview on sdn architectures with multiple controllers. **Journal of Computer Networks and Communications**, v. 2016, n. 1, 2016. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/9396525>. Citado na página 19.

BOSS, Greg et al. Cloud computing. **IBM white paper**, [http://download.boulder.ibm.com/ibmdl/pub/software/dw/wes/hipods/Cloud ...](http://download.boulder.ibm.com/ibmdl/pub/software/dw/wes/hipods/Cloud...), v. 321, 2007. Citado na página 25.

CHOWDHURY, N.M. Mosharaf Kabir; BOUTABA, Raouf. A survey of network virtualization. **Computer Networks**, v. 54, n. 5, 2010. ISSN 1389-1286. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1389128609003387>. Citado na página 30.

DAHLIN, M. Interpreting stale load information. **IEEE Transactions on Parallel and Distributed Systems**, v. 11, n. 10, 2000. Citado 3 vezes nas páginas 28, 32 e 33.

EAGER, Derek L.; LAZOWSKA, Edward D.; ZAHORJAN, John. Adaptive load sharing in homogeneous distributed systems. **IEEE Transactions on Software Engineering**, SE-12, n. 5, 1986. Citado na página 27.

EUGSTER, Patrick Th et al. The many faces of publish/subscribe. **ACM computing surveys (CSUR)**, ACM New York, NY, USA, v. 35, n. 2, p. 114–131, 2003. Citado na página 35.

FETTE, I.; MELNIKOV, A. **The WebSocket Protocol**. [S.l.], 2011. Disponível em: <https://datatracker.ietf.org/doc/html/rfc6455>. Citado na página 26.

gRPC. **Introduction to gRPC**. 2024. Disponível em: <https://grpc.io/docs/what-is-grpc/introduction/>. Citado na página 26.

GUDGIN, Martin et al. **SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)**. [S.l.], 2007. Disponível em: <https://www.w3.org/TR/soap12-part1/>. Citado na página 26.

HAMDAN, Mosab et al. A comprehensive survey of load balancing techniques in software-defined network. **Journal of Network and Computer Applications**, v. 174, p. 102856, 2021. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804520303222>. Citado na página 18.

HAPROXY. **HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer**. 2025. Acessado em: 27 de março de 2025. Disponível em: <https://www.haproxy.org/>. Citado na página 12.

JANE, Kimberly. Scalability issues in database systems: Strategies for scaling databases horizontally and vertically. 03 2025. Citado na página 16.

KIM, Junyeop; WON, Youjip. Dynamic load balancing for efficient video streaming service. In: **2015 International Conference on Information Networking (ICOIN)**. [S.l.: s.n.], 2015. p. 216–221. Citado na página 13.

MA, Chen; CHI, Yuhong. Evaluation test and improvement of load balancing algorithms of nginx. **IEEE Access**, v. 10, p. 14311–14324, 2022. Citado na página 12.

MAHMOOD, Ibrahim et al. Web server performance improvement using dynamic load balancing techniques: A review. **Asian Journal of Research in Computer Science**, 06 2021. Citado na página 13.

MEI, Yiduo et al. Performance analysis of network i/o workloads in virtualized data centers. **IEEE Transactions on Services Computing**, v. 6, n. 1, 2013. Citado na página 11.

Microsoft Corporation. **Memory performance counters**. 2023. Referência oficial para os contadores de desempenho de memória no Windows. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/perfctr/memory-performance-counters>. Citado na página 25.

Microsoft Corporation. **PhysicalDisk object**. 2023. Referência oficial para os contadores de desempenho do objeto PhysicalDisk, usado para monitoramento de I/O de disco. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/perfctr/physicaldisk-object>. Citado na página 25.

MIRCHANDANEY, R.; TOWSLEY, D.; STANKOVIC, J.A. Analysis of the effects of delays on load sharing. **IEEE Transactions on Computers**, v. 38, n. 11, 1989. Citado na página 27.

MITZENMACHER, Michael. How useful is old information? **Parallel and Distributed Systems, IEEE Transactions on**, v. 11, 02 2000. Citado na página 27.

Mozilla Developer Network. **REST - MDN Web Docs Glossary**. 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Glossary/REST>. Citado na página 26.

OASIS. **AMQP Version 1.0**. 2012. Disponível em: <https://docs.oasis-open.org/amqp/core/v1.0/os/amqp-core-overview-v1.0-os.html>. Citado na página 27.

OASIS. **MQTT Version 5.0**. 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>. Citado na página 27.

PANDIAN, A. Pasumpon; FERNANDO, Xavier; HAOXIANG, Wang (Ed.). **Computer Networks, Big Data and IoT: Proceedings of ICCBI 2021**. [S.l.]: Springer Nature Singapore, 2022. v. 117. (Lecture Notes on Data Engineering and Communications Technologies, v. 117). ISBN 978-981-19-0897-2. Citado na página 17.

PASSOS, Edenilson Jônatas dos; FIORESE, Adriano. **Balanceamento de Tráfego Entre Servidores de Vídeo MPEG-DASH Em Redes Definidas Por Software**. 2019. <https://doi.org/10.5753/wgrs.2019.7684>. Accessed 15 May 2024. Citado na página 10.

POSTEL, J. **User Datagram Protocol**. [S.l.], 1980. Disponível em: <https://datatracker.ietf.org/doc/html/rfc768>. Citado na página 27.

POSTEL, J. **Transmission Control Protocol**. [S.l.], 1981. Disponível em: <https://datatracker.ietf.org/doc/html/rfc793>. Citado na página 27.

POSTEL, J.; REYNOLDS, J. **File Transfer Protocol (FTP)**. [S.l.], 1985. Disponível em: <https://datatracker.ietf.org/doc/html/rfc959>. Citado na página 27.

PRESTON, Sam. **Finding the Best Servers to Answer Queries—Edge DNS and Anycast**. Akamai Technologies, 2021. Akamai Blog. Disponível em: <https://www.akamai.com/blog/edge/finding-the-best-servers-to-answer-queries-edge-dns-and-anycast>. Citado na página 22.

SHIVARATRI, Niranjana; KRUEGER, Phillip; SINGHAL, Manish. Load distributing in locally distributed systems. **Computer**, v. 25, 01 1993. Citado na página 27.

SODAGAR, Iraj. The mpeg-dash standard for multimedia streaming over the internet. **IEEE MultiMedia**, v. 18, n. 4, 2011. Citado 3 vezes nas páginas 19, 20 e 21.

STOCKHAMMER, Thomas. Dynamic adaptive streaming over http: Standards and design principles. In: . [S.l.: s.n.], 2011. Citado 2 vezes nas páginas 19 e 20.

TAILOR SUSHANT CHOUDHARY, Vinay Jain Hemang. **Rise of NewSQL**. 2014. Accessed: 2025-03-27. Disponível em: https://dlwqtxts1xzle7.cloudfront.net/84529847/pid-212015010-libre.pdf?1650446168=&response-content-disposition=inline%3B+filename%3DRise_of_NewSQL.pdf&Expires=1743081319&Signature=PrYDe1OEWBWnlHmNa8ucCPryddRUjK5rqTYuRcvGMkLt5SWDKJNVrRE585GUmF~7VpfvOR8CwRQ_&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA. Citado na página 11.

THALLAPALLY, Nagaraju. Enhancing distributed systems with message queues: Architecture, benefits, and best practices. **Journal of Electrical Systems**, v. 21, n. 1s, 2025. Citado na página 29.

The Linux Kernel Developers. **I/O statistics fields for block devices**. 2011. Descreve os campos fornecidos pelo arquivo `/proc/diskstats`. Disponível em: <https://www.kernel.org/doc/Documentation/iostats.txt>. Citado na página 25.

The Linux Kernel Developers. **The /proc Filesystem**. 2020. Documentação do kernel sobre a estrutura e os arquivos do sistema de arquivos `/proc`, incluindo `/proc/meminfo`. Disponível em: <https://www.kernel.org/doc/Documentation/filesystems/proc.txt>. Citado na página 25.

TSAO, Shiao-Li et al. Data allocation and dynamic load balancing for distributed video storage server. **Journal of Visual Communication and Image Representation**, v. 10, n. 2, p. 197–218, 1999. ISSN 1047-3203. Disponível em: <https://www.sciencedirect.com/science/article/pii/S104732039904200>. Citado na página 14.

VAKALI, A.; PALLIS, G. Content delivery networks: status and trends. **IEEE Internet Computing**, v. 7, n. 6, 2003. Citado 2 vezes nas páginas 23 e 24.

WANG, Zheng; HUANG, Jun; ROSE, Scott. Evolution and challenges of dns-based cdns. **Digital Communications and Networks**, v. 4, n. 4, 2018. ISSN 2352-8648. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2352864817300731>. Citado na página 23.

WU, Song et al. A large-scale study of i/o workload's impact on disk failure. **IEEE Access**, v. 6, 2018. Citado na página 11.

GLOSSÁRIO

Anycast: Técnica de rede onde a mesma faixa de endereços IP é utilizada em múltiplos servidores em locais diferentes. As requisições são roteadas para o servidor topologicamente mais próximo do cliente, melhorando a performance e a resiliência.

Broker: Em sistemas distribuídos, um intermediário que gerencia a comunicação entre diferentes componentes de software. Ele recebe mensagens de um publicador e as direciona para os assinantes interessados.

Buffer: Uma área de memória temporária usada para armazenar dados enquanto eles estão sendo movidos de um lugar para outro, compensando diferenças de velocidade entre o produtor e o consumidor dos dados.

Cache: Componente de hardware ou software que armazena dados para que futuras requisições por esses mesmos dados possam ser atendidas de forma mais rápida; os dados armazenados em um cache podem ser o resultado de uma computação anterior ou uma cópia de dados armazenados em outro lugar.

Cloud: Modelo de computação que permite o acesso sob demanda, via rede, a um conjunto compartilhado de recursos computacionais configuráveis (ex: redes, servidores, armazenamento, aplicações e serviços).

Contêiner: Uma unidade padrão de software que empacota o código e todas as suas dependências para que a aplicação seja executada de forma rápida e confiável de um ambiente computacional para outro.

Cookies: Pequenos arquivos de texto que os sites enviam para o navegador de um usuário para armazenar informações sobre sua sessão, preferências ou estado de autenticação, permitindo uma experiência de navegação personalizada.

Datacenter: Instalação física centralizada que abriga a infraestrutura de TI de uma organização, incluindo servidores, sistemas de armazenamento de dados, equipamentos de rede e outros componentes necessários para operar e gerenciar aplicações e dados.

Docker: Plataforma de software de código aberto usada para desenvolver, empacotar e executar aplicações em contêineres, permitindo a separação de suas aplicações da infraestrutura para que você possa entregar software rapidamente.

Ethernet: Família de tecnologias de rede de computadores com fio para redes locais (LANs), definindo o cabeamento e os sinais elétricos para a camada física, e um formato de pacote e protocolo para a camada de enlace de dados do modelo OSI.

Failover: Processo de comutação automática para um sistema ou componente de backup em caso de falha do sistema primário, garantindo a continuidade do serviço com o mínimo de interrupção.

Firewall: Dispositivo de segurança de rede que monitora o tráfego de rede de entrada e saída e decide permitir ou bloquear tráfegos específicos com base em um conjunto definido de regras de segurança.

Gateway: Nó de rede que conecta duas redes diferentes que usam protocolos de comunicação

distintos, atuando como um ponto de entrada e saída para o tráfego de rede.

Host: Qualquer dispositivo computacional conectado a uma rede que oferece recursos, serviços e aplicações aos usuários ou a outros nós na rede.

Hypervisor: Software, firmware ou hardware que cria e executa máquinas virtuais (VMs). Ele permite que um único computador host suporte múltiplas VMs, compartilhando virtualmente seus recursos, como processador e memória.

Middleware: Software que atua como uma ponte entre um sistema operacional e as aplicações nele executadas, oferecendo serviços comuns e funcionalidades de conectividade que não são fornecidas nativamente pelo sistema operacional.

Mininet: Emulador de rede que cria uma rede de hosts virtuais, switches, controladores e links em um único kernel Linux, permitindo o desenvolvimento, teste e prototipagem de aplicações de Redes Definidas por Software (SDN).

Namespaces: Recurso do kernel Linux que particiona recursos do kernel de forma que um conjunto de processos veja um conjunto de recursos, enquanto outro conjunto de processos vê um conjunto diferente, sendo a base tecnológica dos contêineres.

NetBox: Aplicação web de código aberto projetada para gerenciamento de infraestrutura de rede (IPAM - IP Address Management) e de infraestrutura de datacenter (DCIM - Data Center Infrastructure Management).

OpenFlow: Protocolo de comunicação que permite que o plano de controle de uma rede definida por software (SDN) interaja e gerencie os dispositivos do plano de dados (como switches e roteadores).

Peering: Interconexão voluntária de redes de Internet administrativamente separadas com o propósito de trocar tráfego entre os usuários de cada rede.

Player (aplicação Player MPEG-DASH): Aplicação cliente responsável por requisitar os segmentos de mídia de um servidor HTTP e decodificá-los para reprodução, adaptando a qualidade do vídeo com base nas condições da rede.

Proxy: Servidor que atua como intermediário para requisições de clientes buscando recursos de outros servidores. Ele pode ser usado para filtrar requisições, registrar logs, ou compartilhar conexões.

Python: Linguagem de programação de alto nível, interpretada, de script, imperativa, orientada a objetos, funcional, de tipagem dinâmica e forte, conhecida por sua sintaxe clara e legibilidade.

Remote: Refere-se a um sistema, dispositivo ou recurso que está localizado em um local diferente do usuário ou do sistema principal, sendo acessado através de uma rede.

Round-Robin: Algoritmo de escalonamento de processos ou de balanceamento de carga que distribui as tarefas ou o tráfego de rede de forma sequencial e cíclica entre os recursos disponíveis.

Rust: Linguagem de programação multiparadigma compilada, focada em segurança, especialmente a segurança de concorrência e de memória, mantendo um alto desempenho.

Script: Sequência de comandos armazenada em um arquivo de texto simples, executada por um interpretador, para automatizar tarefas ou configurar um sistema.

Shell: Programa de interface de usuário, geralmente baseado em linha de comando, que permite aos usuários controlar o computador através da inserção de comandos textuais.

Socket: Ponto final de um fluxo de comunicação bidirecional entre processos em uma rede. É definido por um endereço IP e um número de porta.

Stalls: Em streaming de vídeo, uma interrupção na reprodução causada quando o buffer do player fica vazio, forçando o vídeo a pausar enquanto mais dados são baixados.

Startup: O processo de inicialização de um sistema computacional ou de uma aplicação, envolvendo o carregamento do sistema operacional, serviços e programas necessários para sua operação.

Streaming: Tecnologia de transmissão contínua de dados, geralmente áudio e vídeo, de um servidor para um cliente, permitindo que a reprodução comece antes que todo o arquivo tenha sido baixado.

Switches: Dispositivos de rede que operam na camada de enlace de dados (camada 2 do modelo OSI) para conectar múltiplos dispositivos em uma mesma rede local (LAN), encaminhando pacotes de dados apenas para o destinatário específico.

WebSockets: Protocolo de comunicação que proporciona canais de comunicação full-duplex sobre uma única conexão TCP, permitindo a interação em tempo real entre um navegador e um servidor web.

Windows: Família de sistemas operacionais gráficos desenvolvidos, comercializados e vendidos pela Microsoft, amplamente utilizados em computadores pessoais.